Journal of King Saud University – Computer and Information Sciences 35 (2023) 101542

HOSTED BY

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

BV-ICVs: A privacy-preserving and verifiable federated learning framework for V2X environments using blockchain and zkSNARKs

Abla Smahi [a], Hui Li [a,b,*], Yong Yang [c], Xin Yang [a], Ping Lu [d], Yong Zhong [e], Caifu Liu [f]

[a] Shenzhen Graduate School, Peking University, Shenzhen 518055, Guangdong Province, China
[b] Peng Cheng Laboratory, Shenzhen 518055, Guangdong Province, China
[c] School of Mathematics and Big Data, Foshan University, Foshan 528000, Guangdong Province, China
[d] Zhongxing Telecom Equipment (ZTE) Corporation, Shenzhen 518057, Guangdong Province, China
[e] School of Electronic Information Engineering, Foshan University, Foshan 528000, Guangdong Province, China
[f] Jiangxi Caiqi Software Technology Co. Ltd, , Jiujiang 330000, Jiangxi Province, China

ARTICLE INFO

Article history:
Received 8 September 2022
Revised 1 March 2023
Accepted 26 March 2023
Available online 11 April 2023

Keywords:
V2X
Federated learning
Blockchain
Edge computing
Verifiable computation
zkSNARKs
PoV

ABSTRACT

As part of vehicle to everything (V2X) environments, intelligent connected vehicles (ICVs) generate a large amount of data, which can be exploited securely and effectively through decentralized techniques such as federated learning (FL). Existing FL systems, however, are vulnerable to attacks and barely meet the security requirements for real-world applications. If malicious or compromised ICVs upload inaccurate or low-quality local model updates to the central aggregator, they may reduce the accuracy of the global model, thereby reducing drivers safety and efficiency. This paper aims to alleviate these concerns by presenting BV-ICVs, a blockchain-enabled and privacy-preserving FL framework for ICVs in an edge-envisioned V2X environment. This system uses Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zkSNARKs) verification that is compiled as smart contracts to prevent malicious, compromised or even rational ICVs from uploading unreliable, erroneous or low-quality model updates. The verification process is embedded within the consensus of the underlying permissioned blockchain, which maximizes both the efficiency of the process and the utilization of computer resources. As demonstrated by discussions, security analysis, and numerical results, BV-ICVs reduced data poisoning attacks and increased the privacy protection and accuracy of FL.

1. Introduction

With the development of infotainment systems and autonomous driving technologies, vehicles are becoming increasingly sophisticated and less reliant on human control. V2X, which stands for 'vehicle to everything', is the umbrella term for the car's communication system, which allows for the quick and efficient exchange of large amounts of data, including high-definition videos, through high-bandwidth and low-latency connections. This data can be used to enhance the driving experience by providing real-time information about the road environment, other vehicles, and potential hazards. For example, cameras mounted on a vehicle can capture high-definition video of the road ahead, and this video data can be transmitted to other vehicles in the network, allowing them to create a shared view of the road environment and make more informed driving decisions An and Wu (2020). So, Intelligent Connected Vehicles (ICVs) are vehicles that are equipped with advanced technologies that enable them to sense, communicate, and respond to their environment Wang et al. (2022), Özdemir and Hekim (2018). According to industry experts, ICVs will on an average churn out 4,000 GB of data per day. The effective utilization of this massively available data opens the door for modern techniques like machine learning (ML), federated learning (FL) and artificial intelligence (AI) to make the system self-adaptive Tian et al. (2018). The nature of FL or ML for AI-enabled ICVs will undoubtedly develop gradually over time and become more complex, surpassing and challenging the computing power of a single vehicle. In this regard, two main directions are evolving; namely, the centralized and the distributed approaches. Manufacturing

* Corresponding author at: Peking University, Shenzhen Graduate School, Lishui Road 2199, Nanshan District, Shenzhen 518055, China.
E-mail address: lih64@pkusz.edu.cn (H. Li).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2023.03.020
1319-1578/

companies are increasingly adopting the centralized method of training, which relies heavily on a central server to perform the process. The entire learning system is vulnerable to the common single-point-of-failure attack if the central server is compromised. ICVs might potentially experience significant privacy loss as a result. Also, large amounts of data from ICVs must be transmitted over the network to the server, which can result in network latency and bandwidth constraints. The problem can be exacerbated by the increasing number of ICVs overloading the server with large amounts of data, thereby degrading its scalability. Data transmission can quickly drain the battery of some types of ICVs with power constraints. There is a growing need for the distributed approach, and FL is shown to be a promising workaround in this regard Pokhrel and Choi (2020). Using FL-based approaches minimizes data sharing by uploading only locally trained model updates, primarily gradient parameters and learning weights. However, building the global model requires a centralized server (aggregator). Unfortunately, although FL can be performed in a decentralized way, the model parameters stored on each local device are neither synchronized nor verified. Therefore, single-point failure issues towards storage still remain Qu et al. (2023). In light of the safety–critical nature of the data and systems involved, privacy, security, and verifiability are imperative. ICVs rely on high-precision algorithms and models to make real-time decisions. Thus, if malicious ICVs upload false or low-quality local model updates to the central aggregator, the global model's accuracy will degrade, and the consequences could be severe for both passengers and the ICV Bachute and Subhedar (2021).

For both centralized and distributed approaches, V2X networks' data contain private information, such as vehicle location and user preferences, that relate to drivers or passengers. Chen et al. (2022). Yet, a considerate collaborative learning framework is strictly required to guarantee its accuracy and reliability. For such frameworks, verifiable computation is an essential building block. The process refers to the validation of the accuracy and quality of the individual models that are trained on each participating ICV. The objective of local model verification is to identify any errors or disqualifications in the local models before they are aggregated and combined into the final global model. Since both approaches have their advantages and disadvantages, the choice will be determined by the specific requirements of the system. This includes the desired level of security and privacy, the size of the dataset, and the computational resources available. It is necessary to look for decentralized solutions with features that enhance security in our scenario of an ICV network, since accuracy and security problems can prove fatal. This is one of the reasons why blockchain technology is so appealing Bowe et al. (2019), Shrestha et al. (2020). Due to its notable features of decentralization, immutability and traceability, blockchain technology is increasingly adopted in vehicular network-based applications to replace the role of centralized model coordinators. Generally, most existing works in the literature adopt the blockchain to manage local model updates and data sharing among vehicles Kang et al. (2019) or to enable a rewarding mechanism for vehicles to incentivize them to faithfully contribute with their local models to the global learning process Toyoda et al. (2020). Although such roles of blockchain seem promising, existing systems are by no means straightforward to provide reliability, integrity and verification of the computations at the same time, especially when outsourced to third parties such as edge or fog nodes. Very few state-of-the-art works consider addressing these challenges by adopting more efficient consensus mechanisms that are based on verifiable computation approaches such as multi-party computation (MPC)Wang et al. (2020), Zero Knowledge Proofs (ZKPs) or trusted oracles. In Zhu et al. (2022), the authors suggested the use of public verifiable secret sharing to verify the correctness of the transmitted data. However, this solution is inefficient due to the massive on-chain communications. Also, although the correctness of the shares allocated to each user can be publicly verified, there is no guarantee that each participant will submit the correct share. Y.Li et al. Li et al. (2021) proposed a committee consensus mechanism to validate local gradients before appending it to the chain. This consensus mechanism is vulnerable to attacks if the malicious percentage reaches 50% and it requires nodes to be switched on throughout the learning process. In other words, it does not apply to vehicular environments where the involved nodes are mobile and suffer from dropouts. Other trusted oracles solutions deploy hardware settings like trusted execution environments to prove the correctness verification against MPC-based cryptography schemes such as fully homomorphic encryption Wenxiu et al. (2021), Fang et al. (2022). However, relying solely on the assurances offered by hardware security mechanisms is arguable due to the emerging attacks such as micro-architectural cache side-channel attacks and transient execution attacks.

Finally, three major hurdles in blockchain-based FL are unavoidable. First, privacy protection must be strengthened while maintaining data utility. Second, to prevent hostile ICVs and edge servers from faking aggregate results, an efficient verification mechanism is required. Third, FL's high communication overhead remains a bottleneck, particularly when combined with blockchain. Whilst workarounds to some of the difficulties have been presented Xu et al. (2020), Qi et al. (2021), there is no extant work in the blockchain-based FL that addresses all three challenges at the same time using blockchain-based solutions. In this research, we propose a blockchain-based FL approach that is both private and verifiable. We design our privacy protocols and verification techniques to enable trustworthy FL among untrusted actors by using the blockchain as the underlying architecture of FL. The following are our major contributions:

1. We begin with introducing a federated SVM, in which the participating ICVs utilize their data to complete local model training and transmit local model updates rather than directly transferring private information, hence, maintaining privacy.
2. To foster the FL application in a decentralized manner, we use a consortium blockchain. The role of the central server is replaced with a collection of trustworthy consensus nodes to handle all of the local model updates provided by participating ICVs. To accomplish so, we take a consensus protocol from the literature Li et al. (2020) and tweak its architecture to make it more trustworthy and applicable to our application case.
3. To the best of our knowledge, this is the first work to suggest employing zkSNARKs to verify local model updates. By doing so, we ensure that only validated models are utilized to generate global models, assuring the FL model's correctness.
4. Our security analysis and experimental results show that the proposed solution effectively prevented data poisoning threats and improved the FL's model privacy protection.

We have designed and instantiated our architecture for using support vector machine (SVM) classification as a FL model, and we have improved the PoV to be used as a consensus mechanism. Our implementation utilizes SNARKs implemented in Zokrates, an open-source tool, for creating and verifying ZKPs using zkSNARKs, as well as smart contracts deployed on an Ethereum virtual machine (EVM) implemented in Solidity. By implementing and testing our system, we have demonstrated that our architecture can achieve reasonable performance and scalability. In addition, we have gained valuable insights into how the combination of FL, ZKPs, and blockchain can be applied to more complex machine learning (ML) models, such as those found in smart vehicular environments. Our FL system integrates several emerging technologies

in a novel way and provides a solution that shows that requirements for integrity, privacy and fairness can be met in practical settings, improving the real-world applicability of existing FL approaches. Our goal is to overcome relevant barriers to FL's application, which may not be appropriate for all use cases. The developed FL system has been designed to be scalable, to ensure integrity, and to provide clients with fair incentives in exchange for their participation in the system.

The remainder of this paper is organized as follows: Section 2 presents the related literature. Section 3 provides a short description of the preliminaries. Section 4 introduces the BV-ICVs design and the corresponding construction details are proposed in Section 5. In Section 6, we discuss the design idea and the security analysis of the proposed system. A performance evaluation is given to demonstrate the proposed framework in Section 7. Finally, some conclusions and future works are presented in Section 8.

2. Related works

In this section, we examine three verifiable computation techniques based on blockchain: trusted Oracles as an already commercially used method, ZKPs, and MPC as the most sophisticated methods.

2.1. Trusted oracles

Incentive-driven off-chain computation (IOC) is a game theory-based method to oracle-based off-chaining. Instead of using cryptographic techniques, an IOC system uses incentive and punishment systems to ensure that computational results are valid. Essentially, the same calculation is replicated across a number of oracle nodes. When all nodes have completed their task, they submit their results to vote on the proper outcome. In the end, the majority vote is accepted as the only solution. IOC systems use an incentive strategy to enforce true behaviour, as their name implies Heiss et al. (2019), Eberhardt and Heiss (2018). However, IOC approaches are limited due to their requirements for an honest majority and the absence of cryptographic guarantees. Some blockchains and associated technologies use various methodologies that rely on trusted execution environments, for example, Intel's SGX Costan and Devadas (2016). Trusting the hardware provider, on the other hand, could be a hazardous bet. Moreover, they are vulnerable to many side-channel attacks such as microarchitectural cache side-channel attacks and transient execution attacks. The IOC is considered to be superior in high-speed ad hoc networking environments because it promotes cooperation and prevents malicious behavior in a dynamic and decentralized environment. Both the nodes and the serving network must meet certain key requirements and features in order for the IOC method to be effective in a high-speed ad hoc network. These may include low latency, high bandwidth, efficient data routing, and robust security mechanisms to prevent unauthorized access and tampering. A potential solution to meet these requirements would be the use of distributed artificial intelligence or multi-agents, which can improve network performance and efficiency by enabling real-time data processing and decision-making at the edge Calvaresi et al. (2019).

2.2. Zero knowledge proofs

Is it possible to demonstrate knowledge of a truth without revealing it? The goal of zero-knowledge proofs (ZKPs) is to solve this fundamental difficulty. Surprisingly, the idea for this technology extends all the way back to the 1980s. A ZKP is informally defined by three main characteristics, namely completeness, soundness, and zero-knowledge. Nowadays, additional functionalities that are extremely desirable for blockchain applications are supported via a new class of zero-knowledge proof: zkSNARKs Ben-Sasson et al. (2014). zkSNARKs were first used by Zcash Ben Sasson et al. (2014) to mask transaction data for their cryptocurrency, but they are now being used by a growing number of other blockchain projects. For instance, for ZK-Rollups ZK-, Ethereum developers intend to greatly boost transaction throughput by putting expensive computations outside the chain. Pinocchio Parno et al. (2013) is another built-in system for verifying general computations using cryptographic assumptions. A client uses Pinocchio to characterize her computation by creating a public evaluation key; this configuration is proportionate to evaluating the computation only once. The worker then evaluates the calculation on a specific input and generates a proof of correctness using the evaluation key. Besides zkSNARKs, there exist other non-interactive ZKPs such as zk-STARKs Eli et al. (2019) and Bulletproofs Bunz et al. (2018a). During the proving phase, zk-STARKs are faster than zkSNARKs, but they are slower during the verification phase. Bulletproofs are a little smaller than zk-STARKs, but they are a lot bigger than zkSNARKs. In terms of proof size, proving time, and verification time, different protocols have distinct limits. When deployed as part of the smart contract execution process, zkSNARKs have the ability to solve many computational overhead and network congestion using proofs of computation Simunic et al. (2021). This is because they avoid the repeated full computations by each network node during the verification process.

2.3. Secure multiparty computation

In a fair world, it is always preferable to have computations that are correct, private, and without a single point of failure. Such an ideal future is promised by secure MPC. MPC encompasses a number of distinct security protocols that all work on the same basic premise: participants supply secret inputs that are disguised using safe obfuscation algorithms, and then execute operations on these values without disclosing them. The protocol's ultimate result is made public at the end. Linear secret sharing and fully homomorphic encryption are the most widely used MPC technologies in the verifiable computation context. For example, SPDZ Damgård et al. (2012) was introduced as a new secure multiparty protocol that uses additive secret sharing to protect general-purpose calculations. The protocol can handle numerous joint parties' inputs, but it always returns a single aggregated computation output. An embedded cryptographic MAC ensures the integrity of the inputs and any intermediate values obtained during execution. Pandora Noyes (2016) was introduced as an Enigma Zyskind (2018) plugin by the creators of Noyes. They used a lot of the calculation engine, but they contributed a lot to the underlying network code. "Keep" Matt and Corbin (2017) is another system that uses linear secret sharing in a way that it creates a decentralized market for verifiable calculations, bringing together buyers and sellers of computing power. MPC-based verifiable computation approaches, when combined with blockchain technologies are still immature and they lack execution environments.

2.4. Verifiable federated learning

We discuss both blockchain-based and non-blockchain-based verifiable computing approaches for FL, in this section. Authors in Li et al. (2021) proposed replacing exchanging the raw data with other parties with a blockchain-based FL approach. The proposed model uses the Mean Absolute Error (MAE) to provide computation verification. However, the verification model is weak especially in the presence of malicious nodes. That is, the selection of the mining leader is made exclusively based on the MAE values that are returned. Similarly, the proposed system in Qi et al. (2021)

suggested using a specific testing dataset to assess the accuracy of the resulting model, which can lead to high computational overhead. Additionally, using differential privacy to establish a globally optimal trade-off between data value and privacy preservation is difficult, and hence the precision of local model updates is not always guaranteed.

Schemes in the literature provided FL's verifiability without the use of blockchains. For instance, VeriFL Guo et al. (2021) was proposed to overcome the problem of high communication overhead that is related to FL aggregation verifiability by committing to the homomorphic hash value of the local gradients. However, the performance of VeriFL is sensitive to the number of clients dropping out. Fu et al. (2020) proposed VFL, a system that provides verifiability of the correctness of the aggregated gradients. This solution is vulnerable to collusion between the server and the clients and hence, invalidating the verification. It should be mentioned that most of these schemes support only the verification of the aggregated models by the server, whereas for such V2X environments, a two-sided verification is essential.

Although most of known verifiable computation methods, such as ZKPs, were described for years, yet their adoption in real-world applications remains limited. None of these approaches, to the best of our knowledge, have been directly applied to FL. In this study, we suggest a consortium blockchain-based FL framework with enabled verifiability to prove the possession of specific data without the need of full disclosure as well as to enable third parties to verify that specific data has been used in the process of model training. This zkSNARKs verification process is integrated as a building block within the blockchain's consensus which also allows for only miners with good reputation to participate in the consensus and they are regularly changeable to provide dynamicity to consensus mechanism.

3. Preliminaries

3.1. Federated learning

In the field of AI, ML refers to the development of algorithms that allow computers to learn from data and make predictions based on this data without being explicitly programmed to do so. In ML, algorithms can be supervised, unsupervised, or semi-supervised, and they can be applied to a wide range of tasks, including image recognition, natural language processing, and prediction.

FL is a type of ML that enables multiple ICVs or edge devices to learn a common model collaboratively while maintaining their computation and individual data sets. This approach differs from traditional ML, which involves the centralization of data and the training of the model on a single server. As part of FL, training data remains distributed and only model parameters are transmitted to the server. The server is responsible for maintaining and sending the latest version of the global model to the clients. These global models are then used by the clients to train on their own data and to produce updated models that are returned to the server. Eventually, the server aggregates the updated models received from the clients and integrates them into the latest version of the global model. This is typically done using a weighted average of the model updates from the clients. This approach has several benefits, such as preserving privacy, reducing data transmission costs, and enabling the use of edge devices that have limited computing resources Qu et al. (2023). Typically, FL is associated with the following optimization problem Li et al. (2020):

$$\min_{\mathscr{W}_g} F(\mathscr{W}_g) = \min_{\mathscr{W}_g} \left(\sum_{i=1}^{|K|} b_i F_i(\mathscr{W}_g)\right) \tag{1}$$

where $|K|$ is the total number of ICVs, $b_i \geqslant 0, \sum_{|K|} b_i = 1$ is the ICV's relative impact, and $F_i(\mathscr{W})$ denotes the local objective function. A weighted average of local weights $\mathscr{W}_g$ is calculated using federated averaging (FedAvg), an aggregation scheme that computes a weighted average of the local weights $\mathscr{W}_i$. It is assumed that the data used for training the ML model are independent and identically distributed samples (IID). In other words, each training example (X, Y) is assumed to be generated independently of all the other examples, and each example is considered to be drawn from the same distribution. When FedAvg is trained on IID data, it produces results similar to those obtained by centralized learning.

So, ML is a broad field of AI focused on building algorithms that can learn from data. FL is a specific approach to ML that allows multiple devices to jointly learn a model while keeping their data decentralized.

3.2. Local differential privacy (LDP)

In their study, Dwork et al. (2006) have developed a concept called differential privacy (DP) which allows secure analysis of sensitive and private data: Consider a trusted central authority that holds a data set that contains sensitive client information. The principle behind DP is to develop a query function that returns the true answer plus random noise according to a carefully chosen distribution Dwork and Roth (2013). Since this privacy guarantee must hold regardless of any and all sources of background information, randomization is the driving force behind it. To achieve this, it is necessary to understand the input and output spaces of randomized algorithms. Formally, M is a randomized algorithm with domain A and (discrete) range B, mapped to $\triangle(B)$, the probability simplex over B:

$$\Delta(B) = \{x \in \mathbb{R}^{|B|} : x_i \geqslant 0 \text{ for all } i \text{ and } \sum_{i=1}^{|B|} x_i = 1\} \tag{2}$$

Suppose two sets of data, $\mathscr{D}$ and $\mathscr{D}'$, that are either equal or differ only in the presence or absence of one individual, are represented as histograms. The histogram of a universe χ is an object in $\mathbb{N}^{|\chi|}$. As a result, both data sets $\mathscr{D}, \mathscr{D}'$, and $\mathbb{N}^{|\chi|}$ are considered adjacent if, for the ℓ_1-norm, $\|\mathscr{D} - \mathscr{D}'\|_1 \leqslant 1$ holds. Dwork et al. (2006) subsequently define DP as follows:

Definition 1. A randomized algorithm M with domain $\mathbb{N}^{|\chi|}$ and range R satisfies ϵ-differential privacy if for every adjacent data sets $\mathscr{D}, \mathscr{D}' \in \mathbb{N}^{|\chi|}$ and any subset $S \subseteq R$ we have

$$Pr[M(\mathscr{D}) \in S] \leqslant e^{\epsilon} Pr[M(\mathscr{D}') \in S], \tag{3}$$

where $\epsilon > 0$ is a privacy parameter.

DP can be achieved by adding noise according to a Laplacian distribution to a numeric function f:

$$\mathscr{L}(z|0, \lambda) = \frac{1}{2\lambda} \exp\left(-\frac{|z|}{\lambda}\right) \tag{4}$$

By adding Laplacian noise to a function $f(z) = \sum_i z_i$ with $z \in \{0, 1\}$ that a user wants to learn, i.e., the total number of 1's in the data set, $\epsilon - DP$ can be achieved as Dwork and Smith (2010):

$$\tilde{f}(z) = \sum_i z_i + q, \text{ where } q \sim \mathscr{L}\left(0, \frac{\Delta f}{\epsilon}\right) \tag{5}$$

Δf indicates f's sensitivity, i.e., f's maximum difference on data sets that differ only by one element Dwork and Smith (2010). Practical algorithms are available for the provision of DP; and they generally have many characteristics that have an impact on privacy or utility Garrido et al. (2021). Local differential privacy (LDP) is a

special case of DP where confusion (i.e., random perturbation) is carried out locally by ICVs rather than by a central authority. This prevents the central authority from inferring or gaining access to the actual ICVs data. LDP ensures plausible deniability for clients by reducing privacy to the lowest level possible (controlled by ϵ), so users cannot distinguish between $\mathscr{D}$ and $\mathscr{D}'$ with confidence Nguyên et al. (2016).

3.3. Relation

A relation may be thought of as a collection of allowable pairs (x, w) and specifies the relationship between some public input x and some private input w (also referred to as witness) Daniel et al. (2019). R belongs to the set $\{R_\lambda\}_{\lambda\in\mathbb{N}}$, a group of polynomially determinable relations on (x, w), where $w \in D_x$ is the witness and $x \in D_x$ is the public input. R is a valid relation if $R(x, w) = 1$. In Campanelli et al. (2019), a modular approach was proposed whereby a witness domain (D_w) can be divided into subdomains of both committed witness $(d \in D_d)$ and free witness $(w \in D_w)$. D_d can, in turn, be divided into q arbitrary domain $(D_1 \times \ldots \times D_q)$, with $q > 1$ is an arity parameter.

3.4. ZKPs and zkSNARKS

Formally, ZKPs are protocols through which a prover can persuade a verifier of the validity of a statement, without disclosing any information. zkSNARKS, on the other hand, are non-interactive systems that are based on ZKPs but they employ shorter and more efficiently verifiable proofs in a non-interactive fashion. A zkSNARK scheme for a relation R is a tuple of probabilistic polynomial-time algorithms **(KeyGen, Prove, Verify)** as shown below Groth (2016):

- $CRS \leftarrow$ Setup (R) : A relation (R) is entered into the setup process, which outputs a common reference string (CRS). It should be noted that this comprises two keys (pk, vk)—one for proving and one for verifying—that are produced by a key generator **(KeyGen)**.
- $\mathfrak{p} \leftarrow$Prove(CRS, x, w): A proof p is produced by the prover algorithm from an input of a relation (R).
- $0, 1 \rightarrow$ Verify $(CRS, x, \mathfrak{p})$: The CRS, a public input x, and a proof $\mathfrak{p}$ are inputs into the verifier algorithm. If the Verify is persuaded, it outputs 1, else it emits 0.

3.5. Commitment scheme

In order to commit to a value or statement while keeping it secret from others and then making it public, users can utilize a commitment scheme Goldreich (2008). A party cannot modify a value or statement they have committed under a commitment scheme. An algorithm tuple CmS = (Setup; Comt; VerComt) is a commitment scheme if it satisfies the notions of correctness, binding, and concealing. It functions as follows:

- Setup $(1^\lambda) \rightarrow$comkey: Inputs the security parameter and produces a commitment key called comkey. This key includes descriptions of the input space D, commitment space CS, and opening space OS.
- Comt $(comkey, d) \rightarrow (cm, op)$: Accepts a value $d \in D$ as and the commitment key (comkey) as inputs, producing a commitment cm and an opening op.
- VerComt $(comkey, cm, d, op) \rightarrow \theta \in \{0, 1\}$: An opening op, a value d, and a commitment cm are provided as inputs by the algorithm, which accepts $(\theta = 1)$ or rejects $(\theta = 0)$.

3.6. Commit-and-prove zkSNARKS

For a commit-and-prove zkSNARKs (CP-zkSNARKs), in order to demonstrate knowledge of (x, w), a relation $R(x, w)$ must hold with respect to a witness $w = (d, w)$, and d must open a commitment cm_d Campanelli et al. (2019). Fundamentally, a CP-SNARKs for a commitment scheme **(CmS)** and a family of relations $\{R_\lambda\}_{\lambda\in\mathbb{N}}$ is a zkSNARKs for a family of relations $\{R_\lambda^{CmS}\}_{\lambda\in\mathbb{N}}$ thereby:

1. Each relation $\in R^{CmS}$ is introduced by a pair $(comkey, R)$, such that $comkey \in \lambda$ Setup (1^λ) and $R \in R_\lambda$;
2. Every relation spans pairings (x, w) where $x := (x, (cm_j)_{j\in[q]}) \in D_x \times CS_q$ is the statement and $w := ((d_j)_{j\in q}, (op_j)_{j\in q}, w) \in D_1 \times \ldots \times D_q \times OS^q \times D_w$, is the witness and the relation holds iff: $\bigwedge_{j\in[q]} VerComt(comkey, cm_j, d_j, op_j) = 1 \wedge$ $R(x, (d_j)_{j\in q}, w) = 1$

Three algorithms make up a CP-SNARK. **CP= (KeyGen, Prove, VerProof)** and below is how it works:

- KeyGen $(comkey, R) \rightarrow (ek, vk)$
- Prove $(ek, x, (cm_j)_{j\in[q]}, (op_j)_{j\in[q]}, w) \rightarrow \mathfrak{p}$
- VerProof $(vk, x, (cm_j)_{j\in[q]}, p) \rightarrow \theta \in \{0, 1\}$

CP-SNARKs have the benefit of enabling commitments to be created before the generation of zkSNARKs keys for a certain relation.

4. System design

4.1. Notations

We describe the notations used in the system in Table 1 below.

4.2. Threat model

We investigate security concerns from two standpoints: Mobile edge servers and users' vulnerabilities. To begin, we will look at the security threats posed by malicious edge servers. In this case, the butlers and commissioners represent the miners.

- Membership Inference Attack: As a result of their access privileges to the consortium blockchain ledger, the edge servers, including butlers and commissioners, are aware of all of the participating ICVs' local model updates, enabling them to retain parameter information pertaining to each ICV's local model. They can also use the local model's parameter information to undertake membership inference attacks on a ICV's local dataset. The participating ICVs have no way of knowing if the edge server has recorded their parameter data, which is a significant violation of the FL principle.

Second, hostile ICVs may use poisoning assaults to sabotage the global model training's accuracy. Poisoning attacks Tolpegin et al. (2020) can be launched by malicious ICVs in a number of ways, some of which are given below:

- Byzantine Attack: By using the Byzantine attack, a malicious ICV can taint the global model of the blockchain by sending the system inaccurate or poor-quality local model updates instead of genuine ones. The accuracy of the global model will indeed decrease badly.

Table 1
List of notations and descriptions.

Notation	Description
x	Feature vector of a new example that we want to classify
α_j	Lagrange multiplier learned during training
y_j	Class labels of the training examples
x_j	Feature vectors of the training examples
b	Bias term learned during training
$\mathscr{N}(x_j, x)$	Dot product between two vectors x_j and x
$\mathscr{G}(\alpha_j, y_j, n, b)$	Computation of the SVM decision function
$\mathscr{P}(z)$	Function that returns the sign of its input
$R_{\mathscr{N},\mathscr{G},\mathscr{P}}$	Polynomially determinable relations corresponding to the functions $\mathscr{N}, \mathscr{G},$ and $\mathscr{P}$, respectively.
x'	Variable introduced to represent the inner product $x_j^T x$
$y_j \odot x'$	denotes element-wise multiplication
x''	Output of function $\mathscr{G}$
$\mathscr{T}$	Target output of the classification problem
$comkey$	Commitment key
ek	Evaluation key
vk	Verification key
cm	Commitment
op	An opening
w	Witness
q	Arity parameter
d	Subdomain of committed witness
$\mathfrak{p}$	Proof
$\mathscr{M}$	Random subset of high-scoring participating ICVs
$\mathscr{D}t^i$	Local data of ICV i at global round t
E	Number of local epochs to run
η	Learning rate
ϵ	Local differential privacy parameter for adding Laplacian noise to $\mathscr{W}^i_{t,e}$
$\mathscr{W}_i$	Local model of ICV i

- Label Flipping Attack: is one common poisoning attack techniques whereby malicious ICVs manipulate the label information of training samples and hence affecting the quality of local model updates Taheri et al. (2020).

4.3. Consortium blockchain for verifiable and privacy-preserving FL: BV-ICVs

In order to support the current brisk pace of research into the technologies that enable the infrastructure that smart cities within the context of industry 5.0, we propose a system that fully harness the "collective intelligence" from the set of participating ICVs, as shown in Fig. 1, with the help of edge computing and blockchain technology. Users (ICVs), multi-access edge computing (MECN) servers, and blockchain system are the core components of the BV-ICVs framework. These components are all connected through the so called Multi-Identifier Network (MIN) Li et al. (2020), Li and Yang (2021) which provides more security enhancements than the standard IP-based network. Since the focus is largely on information security, it should be noted that discussing specifics regarding the underlying network infrastructure is outside the scope of this work.

Blockchain technology replaces the conventional FL's central server to maintain user privacy, data security, and incentive systems. There are three types of decentralized FLs: fully decentralized FLs, partially decentralized FLs, and hybrid FLs that can be switched between them. As discussed in Section 5, BV-ICVs employ a hybrid consensus algorithm known as PoV, which enables a committee of commissioners and butlers to select the candidate block, based on the verification contract results. Thus, single-point failures, man-in-the-middle attacks, and other types of vulnerabilities are reduced or eliminated. On the other hand, a traditional FL system cannot accommodate incentive mechanisms due to its nature. In this regard, the blockchain plays a significant role by providing an incentive mechanism for FLs. There are a variety of ICVs and devices in an FL system, each with different processing capabilities and data storage capabilities. Participants who have access to better resources and have more accurate training results should be provided with additional benefits to remain motivated to contribute further and behave honestly.

As shown in Fig. 1, BV-ICVs is comprised of three distinct layers: the user layer, the blockchain-edge layer, and the blockchain network layer. In the user layer, ICVs gather data from their V2X environment and store it locally. Using the obtained data as inputs, a ML model is trained using federated support vector machines (Federated SVM), following the Eq. 1. A verification process is essential before a new block can be constructed with records of qualified local model updates. An ICV runs a "verification contract" that uses zkSNARKs to provide proofs of the accuracy of local model updates without disclosing the inputs or calculations. The perturbed local model updates and their accompanying proofs are then uploaded to the blockchain's virtual storage. In response to this, the "commissioner's contract" is triggered, which evaluates the claims and digital signatures of the ICVs prior to signing the new block. The process by which a smart contract triggers another is referred to as "contract invocation". After the butlers generate the legitimate block, the updated block data as well as any relevant rewards or punishments are downloaded by the ICVs.

By integrating zkSNARKs-based verification into consensus procedures within the consortium blockchain, the proposed BV-ICV ensures the reliability of the model updates.

To sum up, BV-ICVs is divided into the following five steps:

- Local ML model building and upload: Each ICV in the system obtains the most recent model. Each ICV learns and trains the local ML model ($Local\_Model_{ICV}$) locally with the data it collects, then computes the local diffrential privacy (LDP) version of the model weights and uploads them to the consortium blockchain system in exchange for a later commensurate reward.
- Verification: Receiving the perturbed local model updates triggers the ICVs' verification contract and together they perform a CP zkSNARKS-based verification. At the end of this stage, the commissioners will check the correctness and the accuracy of the calculations made by each ICV in this phase to validate their claim.
- The commissioners gather all confirmed local ML models and package them into blocks. The newly generated blocks will be added to the end of the blockchain after a consensus has been reached using modified PoV consensus mechanism.
- ML models are downloaded from the blockchain to each ICV, which is updated on a daily basis.
- Every ICV integrates the local ML model collection it has obtained with the global ML model update.
- At the end of each training round, scores and rewards are distributed among the participating nodes for them to be used in the reputation-based selection of the future nodes to participate in the coming training round.

5. Consensus algorithm design and verification process

Section 5 is organized as follows: In Section 5.1, we explain how to set up our system from scratch. Specifically, we discussed the challenges and suggested improvements to the underlying consensus algorithm, which explained the various entities and their roles. Section 5.2 describes the initial training step, which is the training of the local models by the ICVs based on their respective IID data. A complete description of the training process is provided in Algorithm 1, which corresponds to the optimization function given in Eq. 1. Following the steps described in Section 3.2, we also

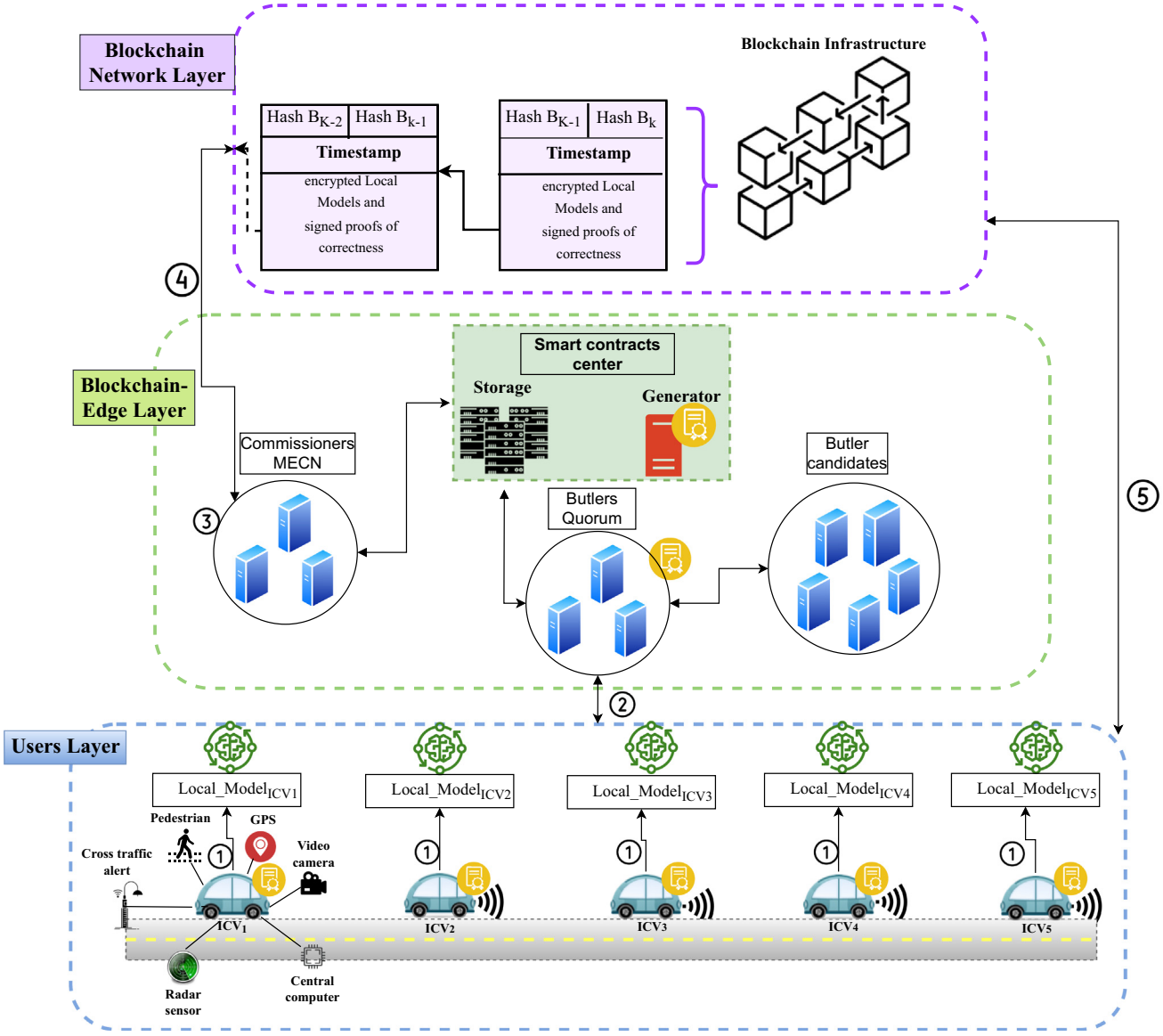

Fig. 1. The proposed system model, ①Training local models, ②Verification, ③Collection of qualified models, ④Verification of candidate blocks, ⑤Download the broadcasted latest local model update.

introduce the perturbation process of the trained local model updates. We then examine the process of verifying local models in Section 5.3. The procedure is divided into four sections: CP-SNARK design and composition (Section 5.3.1), Key generation (Section 5.3.2), Proof generation (Section 5.3.3), and Verification (Section 5.3.4). Section 5.4 provides a description of the candidate block verification process. Lastly, Section 5.5 explains how the global model is constructed. Throughout this section, we will use the notation shown in Table 1.

The consensus procedure among the miners is carried through using a new consensus algorithm that is based on PoV Li et al. (2020) and we improve on it to fit our application scenario with more verifiability features. To establish the most recent global model, only the qualified local model updates are aggregated. This eliminates low-quality local model updates, resulting in a trustworthy FL model.

As opposed to conventional consensus methods, such as computation-intensive proof-of-work, communication complex Byzantine fault tolerance, and unfair proof-of-stake, our improved PoV enables flexible switching of miners without a fixed mining group and partition tolerance. In other words, it ensures service provision regardless of whether the system fails in any part of the network. PoV performs better in terms of block data finality than the delegated proof-of-stake mechanism. We redesign the consensus process to incorporate model update verification into the consensus method, ensuring that FL is secure and dependable. The following are further details on the consensus process (Fig. 2):

5.1. Initialization

Among the entire MECN within the FL environment, four types of roles exist within the adopted PoV: commissioner, butler, butler candidate and ordinary MECNs. A commissioner embodies the role of the delegate MECN miners and it is in charge of verifying the final blocks as well as forwarding them with transactions. Butlers, on the other hand, are responsible for producing blocks whereas the ordinary MECNs are not required to get authenticated to join or exit the network and apply to upgrade their privileges to become butler candidates. However, the proposed PoV in Li et al. (2020) has a number of flaws that can implicate serious security

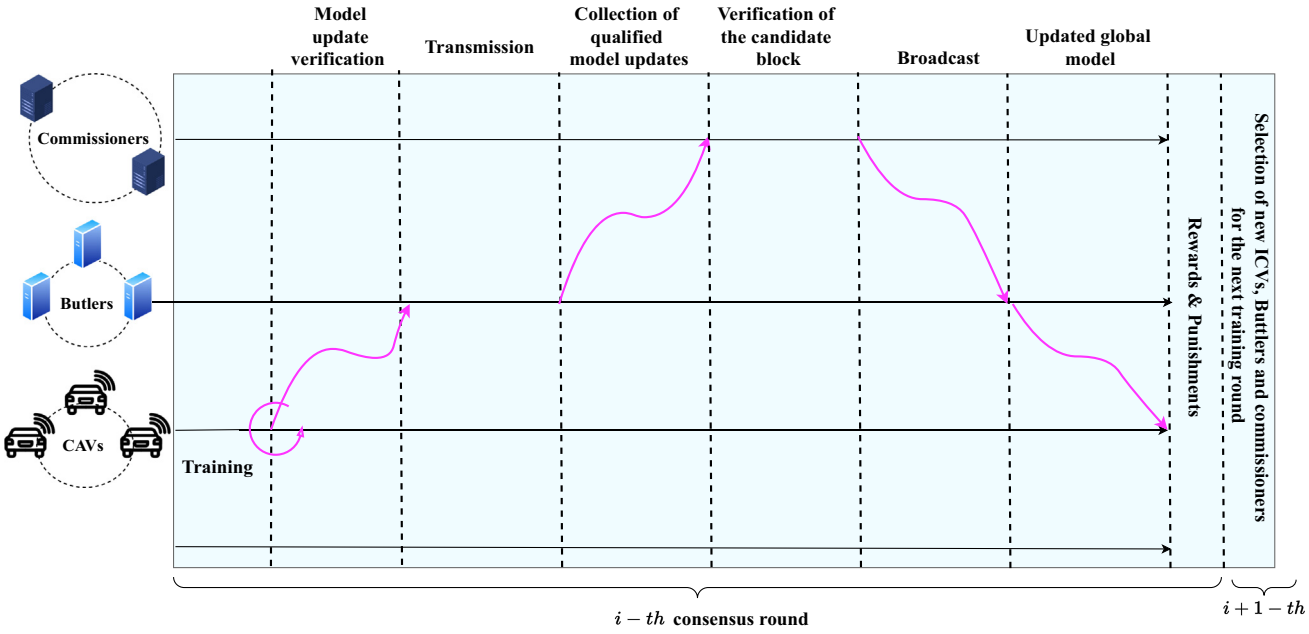

Fig. 2. BV-ICVs consensus process.

flaws, especially in the existence of collusion. These flaws come from the fact that:

- A commissioner joins the miners' group after being introduced by a node that is working in the consortium blockchain. This can pose the entire blockchain under severe security vulnerabilities especially if the node inviting the commissioner is malicious.
- Similar to the first raised problem, commissioners have a plenipotentiary power to recommend and evaluate butlers. In case of collusion between butlers and commissioners for mutual benefits, the entire consortium blockchain can be under security vulnerabilities, notably when malicious butlers are recommended.
- Commissioners do not change throughout different consensus rounds (tenures). That is, the commissioners' group is unchangeable over time and therefore this affects the dynamism of the consensus protocol itself and it also does not incentivize the commissioners to behave honestly since their high and extreme privileges are always guaranteed.
- Finally, there exist a contradiction or a conflict about a node having the right to be a commissioner and a butler at the same time. A butler candidate can become a butler after reaching a voting threshold from commissioners. So, if a node is playing the role of a commissioner and a butler at the same time how can he vote for other butlers.

To solve the above-mentioned problems, we base our roles distributions using reputation-like system. That is, after each round of consensus (Rnd_i), following a well-regulated scoring system, scores are distributed among commissioners, butlers and ICVs in a way that honest entities are awarded scores and malicious nodes are low scored. As such, the recommendation and evaluation of butlers is based on a score-based system or reputation-based system rather than trusting or relying on commissioners or trusted parties.

On the other hand, we abolish the fact that any working node in the consortium blockchain has the ability to introduce commissioners to the mining group. So, for our consortium blockchain system, a global trust authority such as government department is in charge for authenticating the identification of these different nodes. The right to be a commissioner is therefore given to only honest commissioners or butlers, i.e., those with the highest scores. Following such reputation-based concept does not only strengthen the security of the consortium blockchain by removing the roles of trusted parties, but it also incentivizes and encourages the nodes, especially butlers and commissioners, to behave honestly. It should be indicated that within our consensus, the only voting operation is the voting to select butlers from a group of butler candidate by commissioners. In this paper, selected butlers who won the votes are called butlers quorum. After setting the scoring rules, it is now the time for selecting commissioners and ICVs to participate in the FL training. At time t_1, the selection of the ICVs and commissioner MECNs depends on scores from the previous round, while the first training round at t_0 is based on random selections. To this end, assuming the reputation values are integers in a finite set $[0, th_{max}]$, th_{honest} and $th_{malicious}$ are the honest and malicious thresholds within this range respectively. So, if the reputation value (Rep) of any participating node, be it a ICV, butler or commissioner, is larger than the threshold ($rep \geqslant th_{max}$) the node is deemed honest and is eligible to participate in the next round. Similarly, if the reputation of a node ($rep \geqslant th_{max}$) the node is considered malicious and will be de-registered from the consortium blockchain, it can register again.

On the other hand, during this step, the key generator corresponding to the zkSNARKS verification contract runs off-chain to generate the proving and verification keys (pk, vk).

5.2. Local model training

Algorithm 1 uses FL to train a global SVM model on a large dataset distributed across multiple ICVs. Each ICV trains a local SVM model on a random subset of its local data and applies local differential privacy by adding Laplacian noise to the weight vector. The local models are then aggregated and verified on the blockchain using zkSNARKs to ensure the privacy and integrity of the process. Finally, the global model is updated by averaging the final local models using federated averaging method described in Brendan et al. (2017).

Algorithm 1. Verifiable federated SVM with Local Differential Privacy using Laplacian noise addition

Input: $\mathscr{M}$: A random subset of high-scoring participating ICVs
$\mathcal{D}t^i$: The local data of ICV i at global round t
B': The size of the random subset $\mathcal{S}$ of $\mathcal{D}t^i$ to be sampled
E: The number of local epochs to run
η: The learning rate
ϵ: The local differential privacy parameter for adding Laplacian noise to $\mathscr{W}^i_{t,e}$
δ: The local differential privacy parameter for adding Laplacian noise to $\mathscr{W}^i_{t,e}$ (typically set to zero)
$\mathscr{W}_i$: local model of ICV i

Result: Global SVM model

Initialization: Set $\mathscr{W} = 0$;
for *each global round* t **do**
  $\mathscr{M} \leftarrow$ Random subset of high-scoring participating ICVs;
  $\mathcal{D}t^i \leftarrow$ ICV i's local data;
  $\mathcal{D}t^{\mathscr{M}} \leftarrow \mathcal{D}t^i \mid i \in \mathscr{M}$;
  for *each ICV* $i \in \mathscr{M}$ **do**
    Sample a random subset $\mathcal{S}$ of size B from $\mathcal{D}t^i$; **for** *each local epoch* $e \in 1, 2, \ldots, E$ **do**
      // $\ell(\mathcal{S}, \mathscr{W}t, e-1^i)$ is the loss function for training the SVM on subset $\mathcal{S}$ using the current weight vector $\mathscr{W}^i_{t,e-1}$ at epoch e
      // $\nabla\ell(\mathcal{S}, \mathscr{W}t, e-1^i)$ is the gradient of the loss function with respect to $\mathscr{W}^i_{t,e-1}$ at epoch e
      Compute local SVM update: $\mathscr{W}t, e^i \leftarrow \mathscr{W}t, e-1^i - \eta\nabla\ell(\mathcal{S}, \mathscr{W}t, e-1^i)$;
      Apply local differential privacy mechanism by adding Laplacian noise to $\mathscr{W}^i_{t,e}$: $\mathscr{W}^i_{t,e} \leftarrow \mathscr{W}^i_{t,e} + q$ where $q \sim \mathcal{L}\left(0, \frac{\triangle\ell}{\epsilon}\right)$;
    end
    Construct local proof π^i_t for $\mathscr{W}^i_{t,E}$ using zkSNARKs;
  end
  Aggregate local proofs $\pi^i_t \mid i \in \mathscr{M}$ using commissioner's contract;
  Verify aggregated proof on blockchain using verification contract;
  Update global model: $\mathscr{W}_{t+1} \leftarrow \frac{1}{|\mathscr{M}|}\sum_{i\in\mathscr{M}} \mathscr{W}^i_{t,E}$;
end

LDP is a technique used to protect the privacy of individual ICVs' data by adding noise to their local models before they are shared on-chain. The noise is added according to a Laplacian distribution, which is a probability distribution that is commonly used in differential privacy algorithms. The Laplacian distribution has a probability density function given by Eq. 4, where z is a random variable, λ is the scale parameter, and the function is symmetric around zero.

To add Laplacian noise to a function $\not{f}$, as in Eq. 5, a random variable q is generated from a Laplacian distribution with scale parameter $\frac{\Delta\not{f}}{\epsilon}$, where $\Delta\not{f}$ is the sensitivity of the function $\not{f}$, which is a measure of how much the output of the function changes when a single individual's data is added or removed. For example, the sensitivity of a function that computes the sum of a dataset with binary entries is 1.

The Laplacian noise is then added to the function's output, which is $\sum_i z_i$. The resulting noisy output is denoted by $\tilde{\not{f}}(z)$ in Eq. 5. The noise is added in a way that preserves DP, which means that the output of the function after adding noise does not reveal any information about any single individual's data that was used to compute the function.

In Algorithm 1, Laplacian noise is added to the local models of each participating ICV after each epoch of training. Specifically, Laplacian noise is added to the updated weight vector, $\mathscr{W}^i_{t,e}$, which is the result of running a gradient descent step on a subset of the ICV's local data during epoch e of global round t. The amount of noise added is determined by the value of ϵ, which is a parameter that controls the amount of privacy protection provided by the Laplacian noise mechanism. The noise is added to the local model before it is shared with the other participating ICVs.

The anonymity set size refers to the number of ICVs that could have contributed to a particular model update Díaz et al. (2003). It is possible for an adversary to identify a specific user by tracing a particular update if the anonymity set size is too small. In order to increase the anonymity set size, BV-ICVs employs a random shuffling mechanism during the voting and aggregation phases. By using this mechanism, it is ensured that the order in which local model updates are aggregated is random and unpredictable. As a result, it is difficult for an adversary to trace a particular update to a specific user. Using the entropy of the distribution over the possible contributors to a particular model update, we can prove that the anonymity set size in our system is large.

5.3. zkSNARKS-based model update verification

The zkSNARKS code is implemented in our system as a smart contract called the "verification contract". This is beneficial for a number of reasons. First, it allows for the verification of local model updates to be performed in a decentralized and secure manner. A smart contract makes the verification process automated and executed on the blockchain, making it resistant to tampering and manipulation. Secondly, we can enforce synchronization of FL by using a smart contract. As shown in Fig. 3, in BV-ICVs, the verification contract runs on the ICVs and utilizes zkSNARKs to construct proofs of the correctness of the local model updates, based on a proof, a verification key, and public input such as the global SVM model. This local update transaction triggers the zkSNARKS-based verification contract, which utilizes the proving key and verification key generated off-chain. ICVs' private inputs are hashed.

We provide a brief overview of the verification process before diving into the details of the formal computation process. A CP-zkSNARK is a type of zkSNARK in which the prover commits to a set of random values that satisfy the relations to be proved, and then constructs a proof using those committed values. It is then possible for anyone to verify the proof without having to interact with the prover again. We can ensure that, in the context of BV-ICVs, the federated SVM function $f(x)$ is correct by verifying the conjunction of three interconnected relations that decompose it $R_1 \wedge R_2 \wedge R_3$: matrix multiplication, element-wise multiplication, and summation. Essentially, the witness represents a combination of the updates to the local model and the intermediate results of

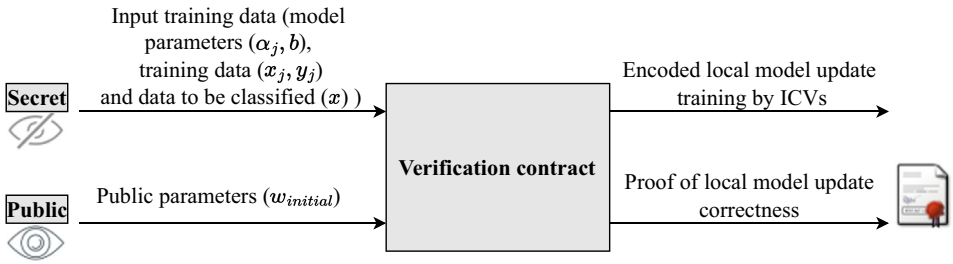

Fig. 3. Verification contract's input and the corresponding outputs.

$R_1 \wedge R_2 \wedge R_3$ that are computed on the local devices. A proof will be generated based on these intermediate results by the prover (i.e., the verification contract) as detailed below. Finally, these proofs are validated by the commissioners in order to ensure that they are accurate and that the verification contract has not been manipulated. As long as the proofs are valid, the commissioners will sign and vote to include the transaction containing the proofs in the next block of the blockchain.

Since we suggested a federated SVM, we analyse the following function, which corresponds to an SVM classifier, to demonstrate the practicality of our proposed system:

$$f(x) = sign(\omega.x + b) = sign(\sum_j \alpha_j y_j (x_j^T.x) + b) \tag{6}$$

In the SVM model, we have a set of training examples, each example has a feature vector x and a corresponding class label y, where x is a d-dimensional vector and $y \in -1, 1$. The goal of the SVM is to find a hyperplane that separates the training examples of different classes with the largest possible margin. In the computation in Eq. (6), the inputs are:

- x: the feature vector of a new example that we want to classify.
- α_j: the Lagrange multipliers that are learned during training.
- y_j: the class labels of the training examples.
- x_j: the feature vectors of the training examples.
- b: the bias term learned during training.

To prove the correctness of the computation (6) above, by considering the result as ($\mathscr{T}$), we can equivalently prove the following statement:

$$\exists(\alpha_j, y_j, x_j, b) : \mathscr{T} = f(x) \tag{7}$$

Three separate sub-computations have been determined after observing various components of the function in 6. Function $f(x)$ can, therefore, be recast as a composition of three different functions, $\mathscr{N}(.)$, $\mathscr{G}(.)$, and $\mathscr{P}(.)$:

$$f(x) = \mathscr{P}(\mathscr{G}(\alpha_j, y_j, \mathscr{N}(x_j, x), b)) \tag{8}$$

$\mathscr{N}(x_j, x)$ is a dot product between two vectors x_j and x. The inputs to this function are the two vectors, and the output is a scalar value.

$\mathscr{G}(\alpha_j, y_j, n, b)$ is the computation of the SVM decision function, which involves computing the weighted sum of the input vectors x_j and their corresponding labels y_j using the learned parameters α_j. The inputs to this function are the learned parameters α_j, the corresponding labels y_j, the scalar value n output by $\mathscr{N}(x_j, x)$, and the bias term b. The output is a scalar value that determines the sign of the decision function.

$\mathscr{P}(z)$ is a function that returns the sign of its input. The input to this function is a scalar value, and the output is a scalar value that is either +1 or −1.

As depicted in Fig. 4, proving the correctness of f(x) requires the zkSNARKs for the conjunction of the three relations $R_{\mathscr{N}} \wedge R_{\mathscr{G}} \wedge R_{\mathscr{P}}$ such as:

$$\begin{aligned}
&R_{\mathscr{N}}(x_j, x, x') = 1 \quad \text{iff } \exists(x', x_j) : \mathscr{N}(x_j, x) = x' \\
&\quad \text{that is } ((x_j^T.x) = x') \\
&\quad \wedge \\
&R_{\mathscr{G}}(\alpha_j, y_j, x', x'') = 1 \quad \text{iff } \exists(\alpha_j, y_j, x', x'') : \mathscr{G}(\alpha_j, y_j, x') = x'' \\
&\quad \text{that is } (\alpha_j, y_j \odot x' = x'') \\
&\quad \wedge \\
&R_{\mathscr{P}}(x'', \mathscr{T}, b) = 1 \quad \text{iff } \exists(x'', \mathscr{T}, b) : \mathscr{P}(x'', b) = \mathscr{T} \\
&\quad \text{that is } (b + \sum_{i=1}^{k} x_i'')
\end{aligned} \tag{9}$$

$R_{\mathscr{N}}$ checks whether there exists a pair of inputs (x_j, x) such that the output of function $\mathscr{N}$ is equal to x', which is a variable introduced to represent the inner product $x_j^T x$. This can be thought of as a matrix multiplication operation, where the input x_j is a row vector and x is a column vector.

$R_{\mathscr{G}}$ checks whether there exist values of α_j, y_j, x', and x'' such that the output of function $\mathscr{G}$ is equal to x'', where $y_j \odot x'$ denotes element-wise multiplication (Hadamard product) between the vectors y_j and x'. This can be thought of as an element-wise multiplication operation.

$R_{\mathscr{P}}$ checks whether there exist values of x'', b, and $\mathscr{T}$ such that the output of function $\mathscr{P}$ is equal to $\mathscr{T}$, where $b + \sum_{i=1}^{k} x_i''$ is the summation of the vector x'' plus a constant b. This can be thought of as a summation operation.

Thus, the conjunction of the three relations in (9) corresponds to the composition of the three functions in $f(x)$, which is a non-linear function. By proving the correctness of this conjunction using zkSNARKs, one can ensure the correctness of $f(x)$ without revealing any information about the inputs x and x_j. By analysing the conjunction in Eq. (9), i.e., $(R_{\mathscr{N}} \wedge R_{\mathscr{G}} \wedge R_{\mathscr{P}})$, one may determine if the function $f(x)$ is correct. In other words, the conjunction in (9) stands for matrix multiplication, element wise multiplication (Hadamard product), and summation respectively. Now that the decomposition of our general integrity statement is presented, it is time to show how effectively the commit-and-prove zkSNARKs are put together:

5.3.1. CP-SNARK design and composition

Here, we show how to construct a proof system for the conjunction of relations to establish the correctness of the given function in (8). To put it another way, we show how to join smaller relation-corresponding CP zkSNARKs to create a new, larger, more sophisticated relation-corresponding CP zkSNARK. The relations must share a commitment slot in order to integrate CP zkSNARKs. The conjunction in (9) can be generally introduced as:

$$\begin{aligned}
&R^{\wedge}_{R_0,R_1}(x_0, x_1, d_0, d_1, d_2, w_0, w_1) = \\
&R_0(x_0, d_0, d_2, w_0) \wedge R_1(x_1, d_1, d_2, w_1)
\end{aligned} \tag{10}$$

d_0, d_1,and d_2 are committed witnesses among which d_2 is the shared commitment between the two relations R_0 and R_1. Theorem (3.2) in Campanelli et al. (2019) states that "if CP_0 is a commit and prove

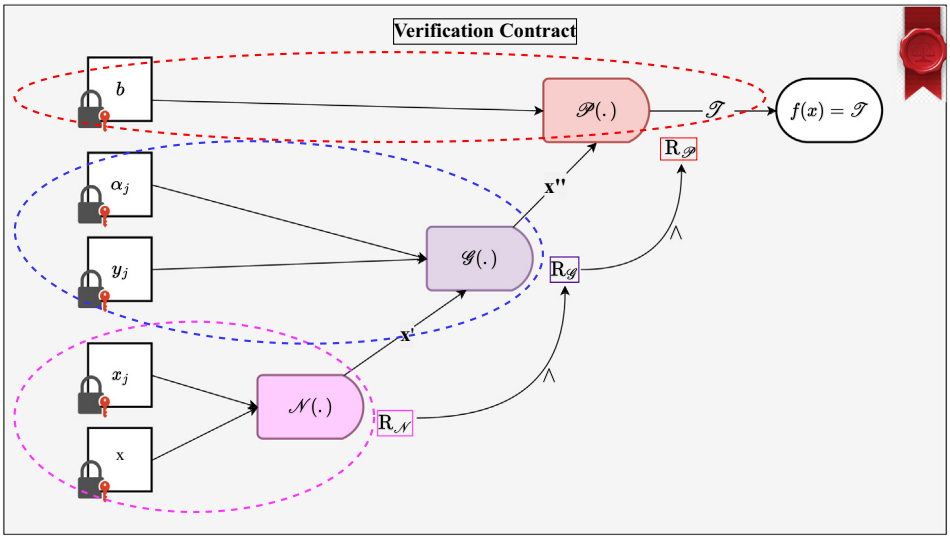

Fig. 4. Verification contract's arithmetic circuit proving the correctness of $f(x)$ which requires the zkSNARKs for the conjunction of the 3 relations $R_{\mathscr{N}} \wedge R_{\mathscr{G}} \wedge R_{\mathscr{P}}$.

zkSNARK for R_0, and CP_1 is a commit and prove zkSNARK for R_1, then there is a commit and prove zkSNARK, $CP^{\wedge}$ for $R^{\wedge}_{R_0,R_1}$."

As we mentioned in Section 3.6 above the composite CP zkSNARKs is made up of three algorithms such that **CP= (KeyGen, Prove, VerProof)**. While KeyGen produces evaluation and verification keys, the Prove algorithm produces evidence that the conjunction is true. VerProof then decides whether to accept or reject.

5.3.2. Key generation

Each existing relation receives one off-chain iteration of the KeyGen algorithm. The related underlying verification contract receives the resulting proving keys. In the verification contract, $R_{\mathscr{N}}, R_{\mathscr{G}}$ and $R_{\mathscr{P}}$ are respectively handled by three underlying interconnected functions within the verification contract F_0, F_1, and F_2.

$$\begin{aligned} &CP^{\wedge\{0,1,2\}}.\text{KeyGen}\ (comkey, R^{\wedge}_{R_{\mathscr{N}},R_{\mathscr{G}},R_{\mathscr{P}}}) \\ &\rightarrow (ek = ek_0, ek_1, ek_2, vk = vk_0, vk_1, vk_2) : \\ &(ek_0, vk_0) \leftarrow CP_0.\text{KeyGen}\ (comkey, R_{\mathscr{N}}) \\ &(ek_1, vk_1) \leftarrow CP_1.\text{KeyGen}\ (comkey, R_{\mathscr{G}}) \\ &(ek_2, vk_2) \leftarrow CP_2.\text{KeyGen}\ (comkey, R_{\mathscr{P}}) \end{aligned} \tag{11}$$

The KeyGen algorithm presented by Eq. (11), for the composite proof system, first performs *KeyGen* for each individual relation $R_{\mathscr{N}}, R_{\mathscr{G}}$, and $R_{\mathscr{P}}$ using the corresponding commit and prove zkSNARKs CP_0, CP_1 and CP_2, respectively. This results in three sets of evaluation and verification keys $ek_0, vk_0; ek_1, vk_1$; and ek_2, vk_2.

Finally, the composite evaluation and verification keys for the conjunction of relations are obtained by combining the individual evaluation and verification keys. The evaluation keys for the composite proof system are the concatenation of the individual evaluation keys, $ek = ek_0, ek_1, ek_2$, and the verification keys are the concatenation of the individual verification keys, $vk = vk_0, vk_1, vk_2$.

The resulting composite evaluation and verification keys are then used by the Prove and VerProof algorithms of the composite proof system, which produce and verify proofs for the conjunction of relations, respectively.

5.3.3. Proof generation

The proof is generated by the Prove algorithm in the CP-SNARK scheme, which takes as input the proving key, the witness, and the statement to be proved. Using the proving key and the witness, the algorithm generates a commitment and a response, which are then used to construct the proof. The result is a proof that is verifiable by anyone who has access to the verifying key. The following proofs apply to the calculations made within the verification contract F_0 and F_1:

$$\begin{aligned} &\text{CP}^{\wedge\{0,1\}}.\text{Prove}\ (ek_0, ek_1, x, (cm_j)_{j\in[0:3]}, (d_j)_{j\in[0:3]}, (op_j)_{j\in[0:3]}) : \\ &\mathfrak{p}_0 \leftarrow \text{CP}_0.\text{Prove}\ (ek_0, x, (cm_0 = cm_{xj}, cm_2 = cm'_x), \\ &(d_0 = x_j, d_2 = x'), (op_0, op_2)) \\ &\mathfrak{p}_1 \leftarrow \text{CP}_1.\text{Prove}(ek_1, x, (cm_1 = cm_{\alpha_j y_j}, cm_2 = cm'_x, \\ &cm_3 = cm''_x), (d_1 = \alpha_j y_j, d_2 = x', d_3 = x''), \\ &(op_0, op_2, op_3)) \end{aligned} \tag{12}$$

in Eq. (12), we have a proof generation algorithm $CP^{\wedge 0,1}.Prove$ The system takes as input two encryption keys ek_0 and ek_1, a value x, and arrays of commitment values $(cm_j)j \in [0:3]$, data values $(d_j)j \in [0:3]$, and opening values $(op_j)_{j\in[0:3]}$. The system generates two proofs $\mathfrak{p}_0$ and $\mathfrak{p}_1$ using two different constraint systems CP_0 and CP_1, respectively. The first proof $\mathfrak{p}_0$ is generated using CP_0 and proves that the value x is consistent with the commitment values $(cm_0 = cm_{xj}, cm_2 = cm'_x)$ and subdomain of committed witness values $(d_0 = x_j, d_2 = x')$, where x_j is the j-th bit of x and cm_{xj} is the corresponding commitment value. The proof also includes two opening values (op_0, op_2). $\mathfrak{p}1$ is similarly generated using CP_1. The opening values $(op_j)_{j\in[0:3]}$ are used to verify the correctness of the commitments. $(op_j)j \in [0:3]$ and $(op_j)j \in [1:3]$ refer to openings of the committed values $(cm_j)j \in [0:3]$ and $(cm_j)j \in [1:3]$ respectively. The proofs generated using these openings provide evidence that the openings correspond to the committed values, without revealing any additional information about the committed values themselves.

Similarly, on F_1 and F_2 support generating $\mathfrak{p}_2$ by the following proofs:

$$\begin{aligned} &CP^{\wedge\{1,2\}}.Prove(ek_1, ek_2, x, (cm_j)_{j\in[1:3]}, (d_j)_{j\in[1:3]}, (op_j)_{j\in[1:3]}) : \\ &\mathfrak{p}_1 \leftarrow CP_1.Prove(ek_1, x, (cm_1 = cm_{\alpha_j y_j}, cm_2 = cm'_x, \\ &cm_3 = cm''_x), (d_1 = \alpha_j y_j, d_2 = x', d_3 = x''), (op_0, op_2, op_3)) \\ &\mathfrak{p}_2 \leftarrow CP_2.Prove(ek_2, x, (cm_{1'} = cm_b, cm_{2'} = cm_{\mathscr{T}}, \\ &cm_3 = cm''_x), (d_{1'} = b, d_{2'} = \mathscr{T}, d_3 = x''), (op_{1'}, op_{2'}, op_3)) \end{aligned} \tag{13}$$

5.3.4. Verification

The VerProof algorithm is responsible for verifying the proofs generated by the Prove algorithm. The verification contract checks

the correctness and accuracy of the calculations made by the prover to validate their claim, which is essentially a proof of knowledge of secret data (w) that satisfies the defined relation R(x,w). The verification contract checks claims made about confidential information for which the system has a public commitment. Once the verification is successful, the verification contract calculates the commitments and stores them in the blockchain repository for further verification by commissioners. During the checking process, the following steps are taken:

$$\begin{aligned}
&CP^{\wedge\{0,1\}}.VerProof(\{vk_0, vk_1\}, (cm_j)_{j\in[0:3]}, \mathfrak{p}_0, \mathfrak{p}_1) \rightarrow b_0 \wedge b_2 :\\
&b_0 \leftarrow CP_0.Verproof(vk_0, x, (cm_0, cm_2), \mathfrak{p}_0)\\
&b_1 \leftarrow CP_1.Verproof(vk_1, x, (cm_1, cm_2, cm_3), \mathfrak{p}_1)\\
&CP^{\wedge\{1,2\}}.VerProof(\{vk_1, vk_2\}, (cm_j)_{j\in[1:3]}, \mathfrak{p}_1, \mathfrak{p}_2) \rightarrow b_1 \wedge b_2 :\\
&b_1 \leftarrow CP_1.Verproof(vk_1, x, (cm_1, cm_2, cm_3), \mathfrak{p}_1)\\
&b_2 \leftarrow CP_2.Verproof(vk_2, x, (cm_{1'}, cm_{2'}, cm_3), \mathfrak{p}_2)
\end{aligned} \tag{14}$$

The Eq. (14) above describes the steps taken by the *VerProof* algorithm to verify the proofs generated by the *Prove* algorithm. The input to the algorithm includes the verification keys vk_0, vk_1, and vk_2 as well as the commitments generated by the prover, denoted as $(cm_j)_{j\in[0:3]}$, and the proof data, denoted as $\mathfrak{p}_0, \mathfrak{p}_1$ and $\mathfrak{p}_2$.

The algorithm first executes the verification for vk_0 and vk_1 in parallel using the first set of commitments and proof data, which results in the output variables b_0 and b_1. Here, b_0 is set to true if the verification using vk_0 is successful, while b_1 is set to true if the verification using vk_1 is successful.

Next, the algorithm executes the verification for vk_1 and vk_2 in parallel using the second set of commitments and proof data, which results in the output variables b_1 and b_2. Here, b_1 is set to true if the verification using vk_1 is successful, while b_2 is set to true if the verification using vk_2 is successful.

The final output of the algorithm is the logical conjunction of b_0 and b_2, which indicates whether the proof has been successfully verified by both verifiers.

5.4. Candidate block verification

After checking the correctness and accuracy by going through the verification of proofs, it is now time for commissioners to validate the new block by verifying the results, the verifications and the corresponding digital signatures. To be considered as a legitimate block, i.e., added to the consortium blockchain, a block must amass at least $\lfloor \frac{N_K}{2} \rfloor + 1$ signatures from commissioners where N_K denotes the number of commissioners. A butler must generate a legitimate block in the specified amount of time. Finally, scores are distributed among all participating nodes, depending on which, after each tenure, a new distribution of roles is proceeded.

5.5. Global model training

The weighted average of all the qualified local model updates is used by all of the participating ICVs to establish their new global models. They accomplish this by downloading the updated block data from the consortium blockchain. For the subsequent round of global model training, the ICVs initialize their models using the updated global model. When the global model training is finished, the ICVs that consistently provided valid local model updates are scored and given a certain amount of tokens as compensation.

6. Discussion of the design idea and security analysis

6.1. Design idea and limitations

We aim to answer the question of how a FL system can achieve verifiability, privacy, and efficient communication simultaneously while remaining practical and scalable. Having identified a business need for these requirements and argued that no architecture has been proposed so far to satisfy them, we described our proposed system in Section 4 utilizing blockchain, ZKP, and LDP. BV-ICVs allows the consensus to verify and vote for the integrity of trained local models, as well as to suggest a method of combining FL with blockchain, zkSNARKS, and LDP. Thus, we extend existing research that has proposed ZKPs for verifying ML model inferences based on previously trained models. Our system also enables any party to verify the proof of the correctness of local model updates, regardless of whether they have access to the ICV's secret inputs. In the verification process, the validity of the proof is checked against a public statement without knowledge of the witness. Cryptographic primitives such as hash functions and elliptic curve cryptography are used to accomplish this, where the ICV creates a succinct proof which is significantly smaller than the witness, and the verifier evaluates the validity of the proof against the public statement within a short period of time.

Although our research has limitations, it also points to future research opportunities. Our proposed system has been designed to be as generic as possible, even though it currently only supports Federated SVM, to permit its adaptation to other classes of ML protocols. The reason for this is that training ML models often involves solving a convex optimization problem whose optimality can be checked locally. CP-SNARKS are not generic constructions that can be applied to any ML protocol, but they may be adapted to different classes of ML protocols with some effort. Essentially, the idea is to encode computation as a series of arithmetic and Boolean operations that can be represented as circuits. Suppose one wanted to apply CP-SNARKs to a different class of ML protocol, such as a deep neural network. In this case, one would need to represent the computation as a circuit of arithmetic and Boolean gates. This process, however, can be quite challenging, especially for complex ML protocols, and requires a thorough understanding of both the ML algorithm and how zkSNARKs work.

Second, in our system, as we previously mentioned, a trusted authority is responsible for setting up the consortium blockchain network used for FL verification. This trusted authority initiates the network and sets up the initial conditions that allow the participating ICVs to begin contributing to the FL process. However, a fully decentralized system would not require a trusted authority to initiate the network. Instead, the network would be established and maintained by a peer-to-peer network of nodes, where each node in the network has an equal role in maintaining the integrity of the network. It will be promising to explore different decentralized consensus mechanisms in order to make our system fully decentralized without sacrificing security and privacy guarantees. Theoretically, zkSNARKs are secure and provide strong integrity guarantees. However, zkSNARKs are susceptible to some attacks, although they are rare and often require knowledge of the system's parameters and architecture to be successful. Invalid/fake proof attacks involve creating invalid proofs that satisfy the verification condition of the zkSNARK. In certain instances, this may occur if the trusted setup process is compromised or the prover uses a different circuit than the one expected by the verifier.

Third, while CP-SNARKs with only a few gates per proof are highly efficient, our system's computational cost may increase

because zkSNARKs have certain inherent limitations, including high computation costs, especially as training data increases. To improve the performance of our system in the future, we are exploring alternative ZKPs systems, such as Bulletproofs Bunz et al. (2018b), which have a lower computational overhead and can be scalable. Furthermore, optimizing zkSNARK implementations and utilizing hardware acceleration techniques can also contribute to improving system performance. The majority of today's ICVs feature powerful hardware capable of handling the computing demands of FL systems. We also anticipate hardware acceleration of ZKP-related operations, specifically proving, to be readily available in the near future, as research in this area is already being conducted, for example, by projects using Ethereum and utilizing ZKPs Alex (2020).

6.2. Soundness

Theorem 1. *Let $\mathscr{M}$ be a FL system consisting of n participants, where each participant updates their local model using stochastic gradient descent with learning rate η and private data. Let $\mathscr{W}^{i}_{t,e}$ denote the weights of the local model of participant i after t global rounds and e local epochs, and let $\mathscr{M}_t$ denote the global model after t global rounds. Then, our commit and prove zkSNARK protocol can verify with high probability that the local model updates $\Delta\mathscr{W}^{i}_{t,e}$ of each participant are correct and can be securely aggregated to update the global model $\mathscr{M}_{t+1}$.*

Proof of Theorem 1. We can prove the soundness of our commit and prove zkSNARKs protocol by showing that an adversary who deviates from the protocol's requirements can only do so with negligible probability.

First, note that each participant i generates a commitment cm_i to their local model update $\Delta\mathscr{W}^{i}_{t,e}$ using the Pedersen commitment scheme. Each participant also generates a non-interactive proof $\mathfrak{p}_i$ of knowledge of the opening op_i of their commitment using our commit and prove zkSNARK protocol.

Next, to aggregate the local model updates, the server generates a commitment to the sum of the commitments $cm_1, \ldots, cm_n$ using the Pedersen commitment scheme. Each participant then generates a proof of knowledge of their contribution to the sum using our commit and prove zkSNARK protocol.

To prove the soundness of our protocol, we assume the existence of an adversary $\mathscr{A}$ who deviates from the protocol's requirements. Specifically, we assume that $\mathscr{A}$ generates a fake commitment cm'_i and a fake proof $\mathfrak{p}'_i$ for participant i, and a fake contribution proof for the global sum.

We can show that the probability of $\mathscr{A}$'s success is negligible by using the properties of the Pedersen commitment scheme and the zero-knowledge property of our zkSNARK protocol.

First, the Pedersen commitment scheme is perfectly binding and computationally hiding. This means that it is infeasible for an adversary to find two distinct openings for a given commitment or to compute the commitment value given only the commitment itself.

Second, our commit and prove zkSNARK protocol is a non-interactive zero-knowledge proof of knowledge. This means that an adversary who deviates from the protocol's requirements can only do so with negligible probability without revealing any information about their secret opening.

Using these properties, we can show that the probability of $\mathscr{A}$'s success is negligible and thus our commit and prove zkSNARK protocol is sound in verifying the computation correctness of local model updates in a federated learning system.

On the other hand, it is important to get an overall estimate of the soundness error. To do so, we need to (1) estimate the soundness error of the ZKP system and (2) to estimate the soundness error of the extractor algorithm.

$$\begin{aligned}\epsilon_{CK} \leqslant Pr[CK \wedge CK' \wedge \pi_{ver}(pk, x, p_0, p_1, p_2) = 1] \\ \leqslant \epsilon_{ZK} + 2q^2 + 2q\epsilon_H\end{aligned} \tag{15}$$

In Eq. 15, ϵ_{CK} is the knowledge soundness error, ϵ_{ZK} is the zero-knowledge soundness error, q is the security parameter, ϵ_H is the hash function collision resistance error, pk is the public key, x is the input to the proof, p_0, p_1, and p_2 are the three proofs generated by the commit and prove zkSNARK. The symbols $\wedge, \leqslant$, and Pr represent logical AND, less than or equal to, and probability, respectively. Eq. 15 provides an upper bound on the probability that an adversary can successfully convince the verifier that they know the computation without actually knowing it.

To estimate the soundness error of the extractor algorithm, we need to prove that the soundness of a ZKP system depends on the fact that no efficient extractor can extract a valid witness from an invalid proof, with non-negligible probability:

$$Pr[\mathsf{extract}(\omega, (\pi_0, \pi_1)) \notin L | (\omega, x) \notin R] \leqslant \frac{\epsilon}{2} + \frac{\delta}{2}$$

where ϵ is the soundness error of the zkSNARK proof, δ is the error probability of the extractor, L is the set of accepting transcripts, R is the set of all possible inputs to the computation, and $\mathsf{extract}$ is the extractor algorithm. We have three proofs π_0, π_1, and π_3 generated by the commit-and-prove protocol. We can define the accepting transcripts L as the set of all possible pairs of inputs (ω, ϕ) such that ϕ is a valid computation of ω using the circuit C and the secret key sk.We can then use the extractor algorithm to extract the witness ω from the proofs (π_0, π_1, π_2) and the public inputs x. The error probability of the extractor δ can be chosen to be negligible, for example 2^{-128}. So, Eq. 16 gives an upper bound on the probability that the extractor fails to extract the witness given two incorrect proofs.

By combining Eq. 15 and Eq. 16, we obtain a bound on the computation knowledge soundness error ϵ_{CK}, which measures the probability that an adversary can convince the verifier that they know the computation without actually knowing it, even when given two incorrect proofs.

6.3. Completeness

Theorem 2. *Our protocol is complete, meaning that an honest prover can convince the verifier of the validity of their claim.*

Proof of Theorem 2. Let x be the witness and let π be the corresponding proof for a statement $stmt$. Let V be the verifier who will verify the correctness of the computation. The protocol proceeds as follows:

- The prover P generates a commitment cm using the commit algorithm, where $cm = \mathrm{Com}(x, comkey)$. The prover P generates a proof π using the prove algorithm, where $\pi = \mathrm{Prove}(stmt, x, cm, w, op, ek)$, and w is the witness for the statement $stmt, op$ is an opening for the commitment cm, and ek is the evaluation key.
- The verifier V receives the statement $stmt$, the commitment cm, and the proof π. The verifier V checks the validity of the proof using the verify algorithm, where $\mathrm{Verify}(stmt, cm, \pi, vk) = 1$ if the proof is valid and 0 otherwise. The verification includes the following steps:

1. Verify that the commitment cm is valid using the commitment key $comkey$.
2. Verify that the opening op is valid for the commitment cm.
3. Verify that the proof π is valid using the evaluation key ek.

Since the prover P is honest and follows the protocol correctly, the proof π generated by the prover is valid. Therefore, the verifier V will accept the proof π as valid, and the statement *stmt* is proved to be true. Thus, our protocol is complete.

6.4. Perfect zero-knowledge

Theorem 3. *The protocol for verifying the computation correctness of local model updates in FL framework using commit and prove zkSNARKs is perfectly zero-knowledge, meaning that an adversary with unbounded computational power cannot learn anything beyond the validity of the statement being proven, even if they interact with the prover and verifier polynomially many times.*

Proof of Theorem 3. Suppose that there exists an adversary $\mathscr{A}$ that can distinguish between a real and simulated proof with non-negligible probability. We will construct a simulator S that can use A to solve the discrete logarithm problem in a bilinear group, which is assumed to be hard.

S takes as input a bilinear group (G, G_T) of order q, a generator g of G, and a public key pk for a commitment scheme. It then runs $\mathscr{A}$ on the real proof, forwarding the proof to A and responding to all queries from $\mathscr{A}$ as follows:

- If $\mathscr{A}$ queries pk for a commitment key, S generates a random commitment key and returns it to $\mathscr{A}$.
- If $\mathscr{A}$ queries pk for a commitment to a value v, S commits to v using the commitment key generated in step 1 and returns the resulting commitment to $\mathscr{A}$.
- If $\mathscr{A}$ queries pk for an opening of a commitment C, S generates a random witness for the committed value and returns it to $\mathscr{A}$.
- If $\mathscr{A}$ queries pk for a proof of knowledge of a committed value v without revealing v, S uses the commit-and-prove protocol to generate a proof of knowledge for v without revealing v.
- If $\mathscr{A}$ queries pk for a simulation of the proof of knowledge of a committed value v without revealing v, S generates a simulated proof of knowledge for a random value v' and returns it to $\mathscr{A}$.

After $\mathscr{A}$ outputs its guess b for whether the proof is real or simulated, S outputs b as well.

Since our protocol is perfect zero-knowledge, the output of $\mathscr{A}$ is indistinguishable from the output of S with non-negligible probability. Therefore, if $\mathscr{A}$ can distinguish between the two with non-negligible probability, then S can be used to solve the discrete logarithm problem in the bilinear group with non-negligible probability as well, which contradicts the assumption that the problem is hard. Therefore, our protocol is perfect zero-knowledge.

6.5. Resistance against membership inference attacks

Our proposed system employs LDP on the local model updates before appending them to the blockchain. This provides a layer of privacy protection against membership inference attacks, as the noise added to each model update makes it difficult for an adversary to distinguish between the updates of different users. This is because the noise added to each user's update will result in the update being slightly different from the original update, making it difficult to distinguish between the updates of different users.

Formally, we can show that our system is resistant to membership inference attacks by proving that the probability of a successful attack is low. Let A be an adversary attempting to perform a membership inference attack on our system. Then, the probability of success can be defined as:

$$Pr[A(\mathscr{D}) = 1] \leqslant \frac{1}{2} + \frac{\epsilon}{2},$$

where $\mathscr{D}$ is the dataset used to train the local models and ϵ is the privacy parameter of the local differential privacy mechanism. This inequality holds for any adversary A and any dataset $\mathscr{D}$. Thus, by setting a sufficiently low value of ϵ, we can make it computationally infeasible for an adversary to perform a successful membership inference attack.

6.6. Resistance against Byzantine attacks

The consensus mechanism in our system is based on PoV, which ensures that only authorized nodes are permitted to take part in the consensus process. It also ensures that each node's vote is weighted based on its reputation and stake in the network. To prevent Byzantine attacks, PoV uses a threshold signature scheme in which each authorized node has a unique private key and a threshold number of nodes must sign a transaction before it can be executed, i.e., at least $\lfloor \frac{N_k}{2} \rfloor + 1$ signatures from commissioners where N_K denotes the number of commissioners. Therefore, a single node cannot maliciously sign a transaction and cause a consensus to be broken.

7. Evaluation results

In this section, we evaluate the performance of BV-ICVs from three aspects: the efficiency (computation and communication complexities), the model accuracy and the computation and communication overhead of the verification process.

7.1. Computation complexity

The computation complexity of the verification protocol can be broken down into the following steps:

1. The verifier computes $\mathfrak{p}_0$ and $\mathfrak{p}1$ using Eq. (12), which involves two calls to the commit-and-prove zkSNARKs CP_0 and CP_1, respectively. Each call to CP_0 and CP_1 involves a single round of interaction between the prover and verifier, where the prover sends a proof to the verifier, and the verifier checks the proof. The computation complexity of each CP_i depends on the complexity of the underlying zkSNARK construction, but typically involves a polynomial evaluation and some cryptographic operations. Let us denote the computation complexity of each CP_i as C_{CP}.
2. The verifier computes $\mathfrak{p}_0, \mathfrak{p}_1$, and $\mathfrak{p}_2$ using Eq. (12) and (13), which involve two calls to the commit-and-prove zkSNARKs CP_0, CP_1, and CP_2 respectively. Each call to CP_0, CP_1, and CP_2 involves a single round of interaction between the prover and verifier, where the prover sends a proof to the verifier, and the verifier checks the proof. The computation complexity of each CP_i depends on the complexity of the underlying zkSNARK construction, but typically involves a polynomial evaluation and some cryptographic operations. Let us denote the computation complexity of each CP_i as C_{CP}.
3. The prover computes the operations op_0, op_1, op_2, and op_3 using the arithmetic circuit described by Eqs. (12) and (13). Since each gate in the circuit involves a multiplication and an addition, and there are $4m$ gates in total, the computation complexity of computing the operations is $O(m)$.
4. The prover computes the commitments cm_0, cm_1, cm_2, and cm_3 using Eqs. (12), which involves $4m$ multiplications and $2m$ additions. The computation complexity of computing the commitments is $O(m)$.

5. The prover computes the witnesses d_0, d_1, d_2, and d_3 using Eqs. (12) and (13). Each equation involves a dot product and a Hadamard product, which can be computed using $2m$ field operations. Therefore, the computation complexity of computing the witnesses is $O(m)$.
6. The prover computes the openings op_0, op_1, op_2, and op_3 using the commitment scheme defined by CmS = (Setup; Comt; VerComt), which involves 4 calls to the Comt algorithm. The computation complexity of each call to Comt depends on the commitment scheme construction, but typically involves some cryptographic operations and polynomial evaluations. Let us denote the computation complexity of each Comt call as C_{Comt}. Therefore, the computation complexity of computing the openings is $4C_{Comt}$.

Therefore, the total computation complexity of the protocol is:

$$C_{total} = 2C_{CP} + 3m + 4C_{Comt}$$

KeyGen was not included in the complexity analysis because it may be done offline and is only executed once per model.

7.2. Communication complexity

Following is a description of the communication complexity analysis:

1. The communication cost of the verifier sending ek_0 and ek_1 to the prover is $O(|ek_0| + |ek_1|)$.
2. The prover computes $cm_0 = cm_{xj}$ and $cm_1 = cm_{\alpha_j y_j}$ using Eq. (12), which requires $2m$ multiplications and m additions. Since each multiplication and addition involves communication of one field element, the communication cost of computing cm_0 and cm_1 is $O(m)$.
3. The prover computes $cm_2 = cm'_x$ and $cm_3 = cm''_x$ using Eq. (13), which requires $2m$ multiplications and m additions. The communication cost of computing cm_2 and cm_3 is also $O(m)$.
4. The prover sends the commitments cm_0, cm_1, cm_2, cm_3 to the verifier, which incurs a communication cost of $O(4m)$.
5. The prover computes the witnesses $d_0 = x_j, d_1 = \alpha_j y_j, d_2 = x'$, and $d_3 = x''$ using Eqs. (12), which involve m dot products and m Hadamard products. Since each dot product and Hadamard product requires communication of one field element, the communication cost of computing the witnesses is $O(m)$.
6. The prover computes the openings op_0, op_1, op_2, and op_3 using the commitment scheme defined by CmS = (Setup; Comt; VerComt), which involves 4 calls to the Comt algorithm. Each call to the Comt algorithm requires communication of a field element from the prover to the verifier, and communication of two field elements from the verifier to the prover (for the commitment cm and the opening op). Therefore, the communication cost of computing the openings is $O(12m)$.
7. The prover sends the witnesses and operations to the verifier, which incurs a communication cost of $O(8m)$.
8. The verifier computes $\mathfrak{p}_0$ and $\mathfrak{p}_1$ using Eq. (12), which involve two calls to the commit-and-prove zkSNARKs CP_0 and CP_1, respectively. Since each call requires communication of a constant number of field elements, the communication cost of computing $\mathfrak{p}_0$ and $\mathfrak{p}_1$ is constant.
9. The prover sends $\mathfrak{p}_0$ and $\mathfrak{p}_1$ to the verifier, which incurs a communication cost of a constant number of field elements.

Therefore, the total communication complexity of the system is:

$$O(|ek_0| + |ek_1| + 6m + 12m + 8m + c) = O(|ek_0| + |ek_1| + 26m + c)$$

where c is a constant that represents the communication cost of the commit-and-prove zkSNARKs. Note that this analysis assumes that the field elements and intermediate results are sent uncompressed, which may not be optimal in practice. Also, the size of the common reference string (i.e., evaluation keys, verification keys) and the volume of data transmitted across different contracts have a substantial impact on how effectively the system works as it scales.

7.3. The model accuracy and the computation and communication overhead of the verification process

The trials and verification's off-chain benchmarks were carried out on a PC running VMware with Ubuntu 16.04 OS and an Intel i7-7660U CPU clocked at 2.50 GHz with 16 GB RAM to assess the performance of the BV-ICVs scheme. For the implementation of the smart contracts, we used ZoKrates, a toolbox for zkSNARKs on Ethereum. Concretely, we contributed precompiled contracts to Ethereum as the verification algorithm's building blocks. In order to generate the proving key and verification key, we execute the generator off-chain. The proving key can then be used to produce a proof off-chain by any prover. Then, using the proof, the verification key, and the public input as input parameters, we execute the general verification algorithm inside of a smart contract. The system's derivative contract is triggered by the results of the verification algorithm for ICVs. The basic imperative ZoKrates language has static scoping and is intended to convert into provable constraint systems. So, we first evaluate the constrained system compiled from ICV's contract vs the commissioners contract respectively.

As can be seen in Fig. 5, the constraint system with the least (695 constraints) was compiled from SC_3, which corresponds to the summations in R_P. This is because additions can be easily agglutinated to a multiplication gate and save several constraints.

Fig. 6 illustrates the requirements for compiling and configuring the ZKPs. Considering that peak resource requirements are of primary concern in intelligent vehicular technology, the maximal memory requirements are provided. The usage of memory over time is also shown in order to determine how long these peak requirements last. Fig. 6 demonstrates that as the batch size increases, the peak memory requirements also increase. The batch size of 40 has the highest peak memory requirement, reaching almost 3 GB. However, this peak requirement is sustained for a short amount of time. Accordingly, depending on the dataset used for training the FL model, this may limit the types of data that can be used. Compilation time and memory intensity are also affected by the input and output dimensions. In every update round, participants in the FL process must compute witnesses and proofs corresponding to global weights and biases for that particular round, as well as the input data used to conduct the training. Figs. 7 and 8 illustrate the resources required for the generation of the witness and proof in terms of time and memory. Clearly, as batch sizes increase, the amount of memory required to generate the proof becomes a bottleneck, especially when the proof has to be generated on an ICV. The memory capacities of L2 and L3 self-driving vehicles in use at the time of writing this manuscript usually exceed 100 GB. The use of 1.8 GB of RAM by an ICV during peaks for computing the proof may have an adverse effect on its other daily tasks.

To test the accuracy and performance of BV-ICVs, we built the FL environment with Python (v-3.6.3), TensorFlow (v-2.4.2) and TensorFlow-federated(v-0.17.0). We used two datasets MNIST[1] and BelgiumTS[2], which are described in the Table 2 blow. There is

[1] MNIST dataset: https://deepai.org/dataset/mnist.
[2] BelgiumTS dataset:https://btsd.ethz.ch/shareddata/.

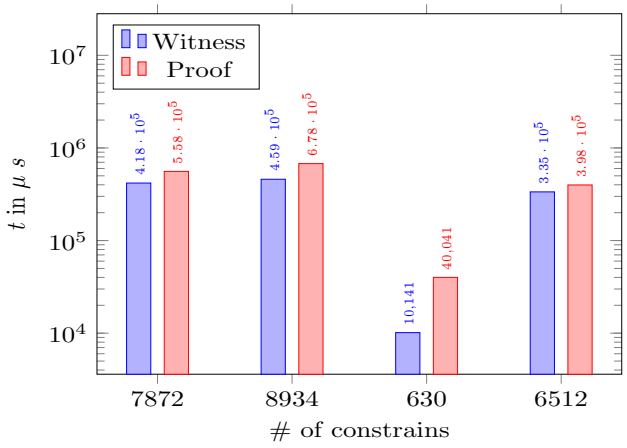

Fig. 5. Average time of witness and proof generation (after 93 execution).

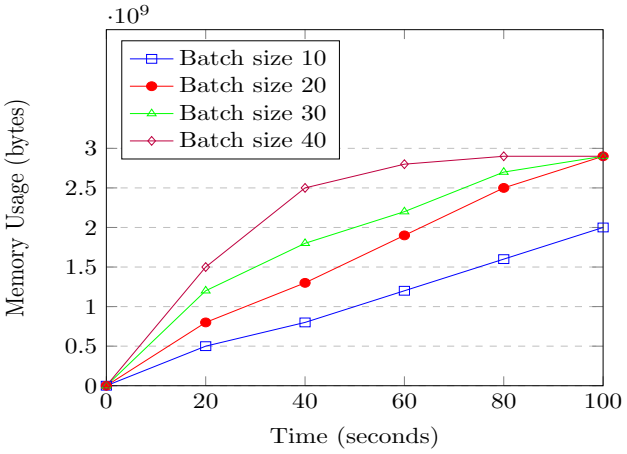

Fig. 6. Memory usage of compilation and setup with different batch sizes.

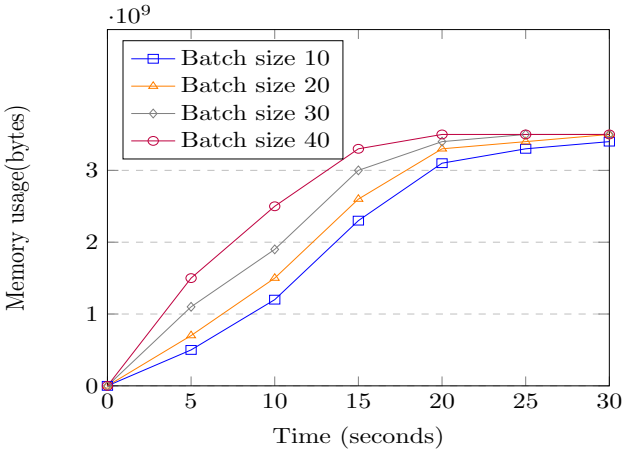

Fig. 7. Memory usage of computing witnesses with different batch sizes.

a chance that not every client will be online at once because of the vehicles' dynamic nature. For training in each round, 20% of the clients are chosen at random. We first simulate a label flipping attack on the two datasets to test the effectiveness of the BV-ICVs model accuracy. This attack involves flipping some ICVs' sample labels to make them appear to be malicious.

The more malicious FL nodes there are, the more of a threat they are to the model's accuracy, as indicated by the comparison

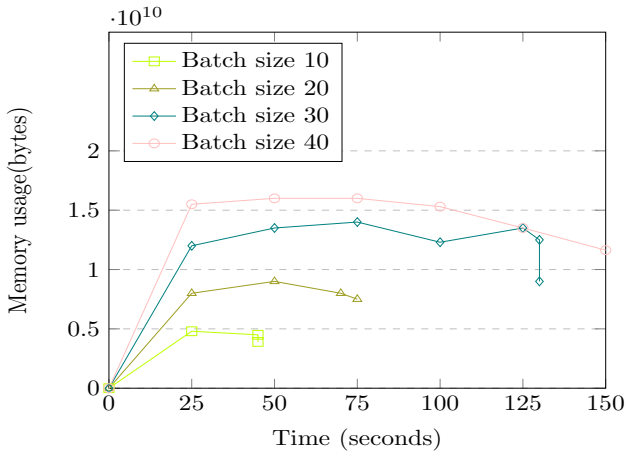

Fig. 8. Memory usage of computing the proofs with different batch sizes.

Table 2
Dataset settings.

Items	Datasets	
	MNIST	BelgiumTS
Size of train/test set	60000/10000	4575/2520
Label classes	10	62

of the two different schemes in Fig. 9 and Fig. 10. It can be observed that the accuracy of the BV-ICVs FL model consistently outperforms that of the standard model without any security enhancements. The performance of the original Federated SVM was drastically reduced by raising the ratio of the flipping attack on the BelgiumTS dataset. This demonstrates how BV-ICVs improves the FL models' accuracy and resilience. To better simulate a V2X environment, we further tested the system's accuracy with an increasing number of clients instead of using a fixed number (10 ICVs) with random ratio of nodes. To do so, we assumed that there are 30 local clients in the system, each with a local dataset obtained by randomly shuffling and evenly splitting the MNIST and BelgiumTS datasets. We set the system's hyperparameters as follows: learning rate = 0.01, mini-batch size = 20, local training epochs = 10. As for the underlying consortium blockchain we again set the numbers of butlers and commissioners as 10 and 8 respectively.

As shown in Fig. 11 and Fig. 12, our findings indicate that the accuracy of the proposed model improves as the number of ICVs (clients) who provide data grows. This is because more data is pooled together, allowing the training approach to improve their model-learning accuracy. Additionally, it is observed that our scheme's model accuracy is comparable to the original federated SVM and outperforms the work presented by Fang et al. (2022). Our proposed model and the original federated SVM achieve best accuracy of 93% and 90%, respectively, on the MNIST dataset at an ICVs clients count of 30, whereas Fang et al. (2022) achieves just 88% accuracy. This difference is due to loss of utility as a result of the addition of random masks to the client's local gradients.

On the other hand, we tested the computation and communication overheads of BV-ICVs on both users (ICVs clients) and blockchain sides. As can be seen from the entire consensus protocol introduced in Section 5, obtaining the global model can be affected by the dropout rates. Although the execution of verification contracts is done off-chain, which mitigates its influence to some extent, phases that require clients to be online, i.e., the verification, candidate blocks verification and global model training, might be affected. Therefore, our scheme's computational overhead is pro-

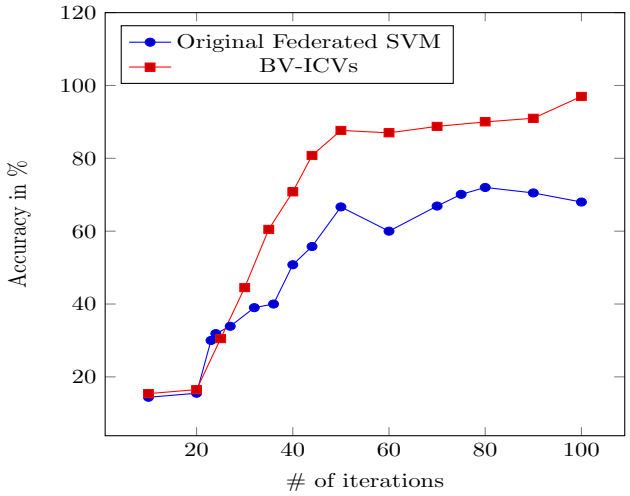

Fig. 9. Comparison of the model's accuracy between BV-ICVs and the original FL with 10 ICVs using the MNIST dataset.

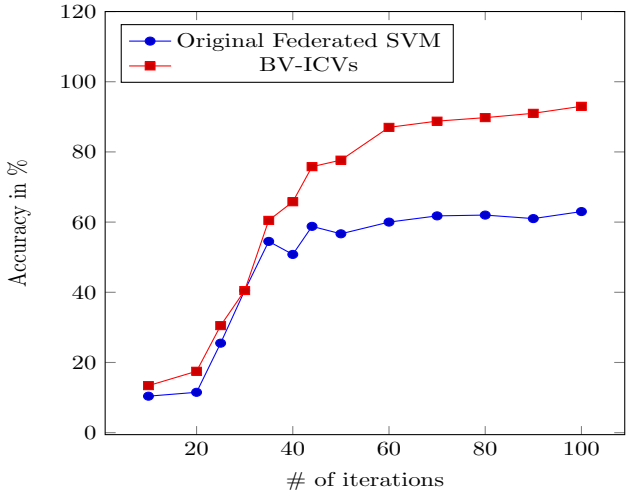

Fig. 10. Comparison of the model's accuracy between BV-ICVs and the original FL with 10 ICVs using the BelgiumTS dataset.

portional to the percentage of nodes that drop out. Now, we test and visualize the average runtime across all phases of our scheme per iteration at varying dropout rates, and then we compare our scheme's runtime to that of Fang et al. (2022) and the original federated SVM.

Fig. 13 depicts the execution time of BV-ICVs in comparison to the original federated SVM and Fang et al. (2022)'s scheme for varying dropout rates. It is evident that: (1) The dropout rate has no impact on the running duration of the original federated SVM since the training of the federated learning model does not account for clients dropping out. (2) The running time of BV-ICVs and Fang et al. (2022)'s schemes is almost double as that of the original federated SVM since the latter just requires gradient aggregation by the central server, but our and Fang et al. (2022)'s schemes incorporate gradient verification and consensus in the blockchain. (3) When the dropout rate reaches or exceeds 10%, our system's running time is less than that of Fang et al. (2022)'s system. The reason for this is that when dropouts occur, the scheme of Fang et al. (2022) demands computationally intensive activities, such as verification of secret shares, reconstruction of private keys for dropped out clients, and creation of redundant random numbers. BV-ICVs, on the other hand, rely on the autonomous re-execution of smart contract code. Of note, for BV-ICVs, With an increase in

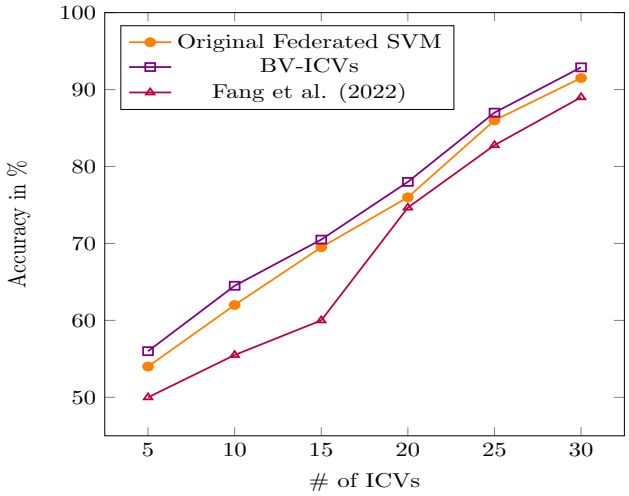

Fig. 11. The accuracy of BV-ICVs under increasing number of ICVs using the BelgiumTS dataset compared with original federated SVM and Fang et al. (2022)'s model.

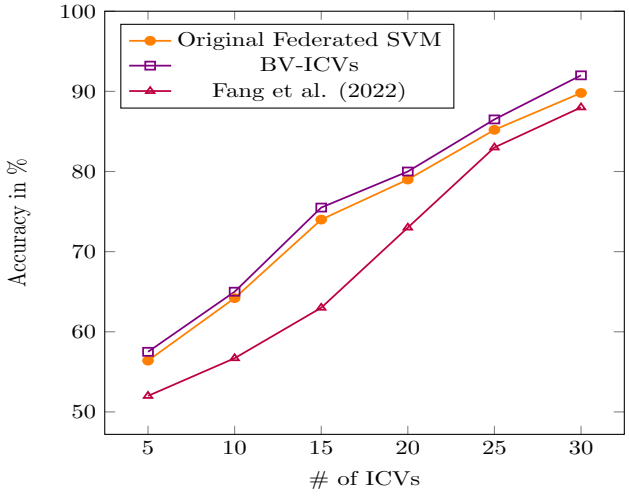

Fig. 12. The accuracy of BV-ICVs under increasing number of ICVs using the MNIST dataset compared with original federated SVM and Fang et al. (2022)'s model.

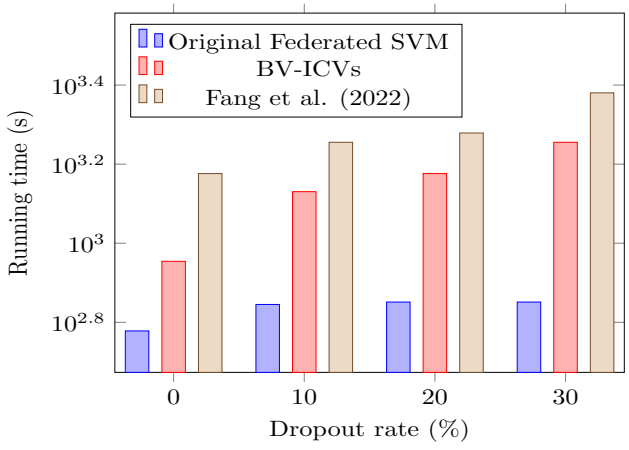

Fig. 13. The running time of BV-ICVs against original federated SVM and Fang et al. (2022)'s scheme under increasing dropout rates using th.e MNIST dataset.

the dropout rate, the running time of the blockchain's side operations drops marginally, since it consists mostly of local model gradient verification. Therefore, the less valid local gradients there are, the less calculation from the blockchain nodes is required. In fact, before the dropout rate reaches 10%, our system's running time is already less than Fang et al. (2022)'s system. This is because BV-ICVs partially performs some operations off-chain, such as training and verification, thereby alleviating the risk of clients leaving the federated learning system before the training round has ended. By contrast, Fang et al. (2022)'s system trains the federated learning system itself to adjust to the dropout of clients, which requires time and computational resources for the system to learn and may not be as efficient as our approach.

Contrarily, on the blockchain side, the increase in dropout rate decreases the communication overhead of block creation and global model training as well as of the local training phase on the clients side. The rationale for this is that the fewer valid local gradients there are, the less data the blockchain and the new block need to receive. Moreover, the communication burden on the blockchain side is significantly larger than on the client side, as commissioners and butlers must receive every client-uploaded data. However, in our system, such nodes are referred to as edge servers (MECNs), which are expected to have sufficient communication capacities, allowing our scheme to be effectively implemented in blockchain-based environments.

Fig. 14 above illustrates the communication overhead incurred by each of the three approaches at varying dropout rates. The communication overhead of our scheme and Fang et al. (2022)'s is larger than that of the original federated SVM, which is defined by the blockchain's communication mechanism. The communication cost of Fang et al. (2022) is smaller than that of BV-ICVs since we utilized off-chain computations and only retrieve the related proofs on-chain using derivative smart contracts. In contrast, the Fang et al. (2022)'s technique employs CRT to compress the local gradient, and all data shares are transferred on-chain, which has a negative impact on the communication overhead. Also, the difference in accuracy, running time and communication overhead can be attributed to the differences in the architecture and approach between the two systems. The original federated SVM is centralized and therefore has more control over the data and the learning process. This can lead to faster computation time and lower communication overhead, as the data can be more easily aggregated and analyzed. On the other hand, BV-ICVs is decentralized and blockchain-based, which introduces additional overhead for verifying local model updates using zksnarks. This can lead to slower computation times and higher communication overhead, but provides additional security and privacy guarantees. Therefore, the trade-off between accuracy and running time/communication overhead ultimately can be accepted when considering the critical nature of V2X federated learning applications.

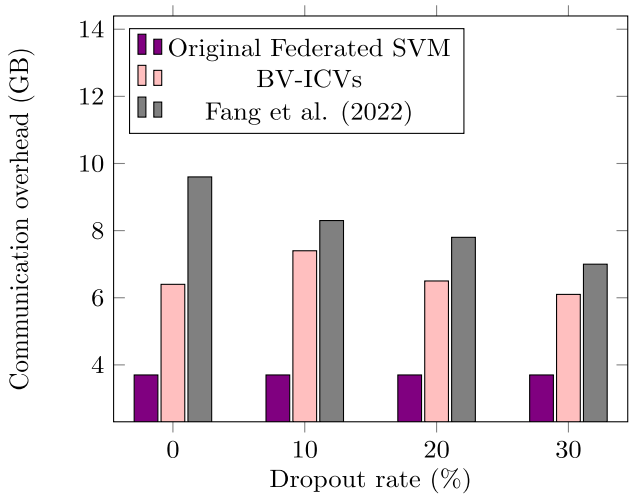

Fig. 14. The communication overhead of BV-ICVs against original federated SVM and Fang et al. (2022)'s scheme under increasing dropout rates using th.e MNIST dataset.

7.4. Testing the underlying Multi-Identifier Network's (MIN) security

As indicated in Section 4.3, BV-ICVs is built on top of a multi-identifier network referred to as MIN Li et al. (2020). MIN architecture is a proposed solution to address the limitations of the IP protocol in terms of centralization, scalability, mobility, and security. MIN allows the simultaneous coexistence of multiple identifiers, including identity, content, geographical information, and IP address, in order to reduce the semantic overload of IP addresses

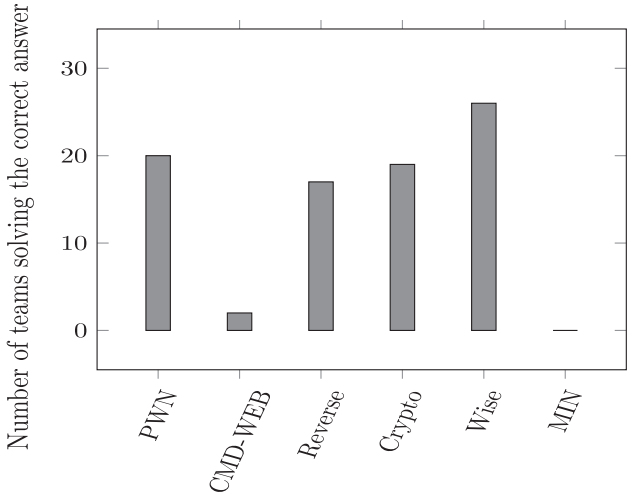

Fig. 15. Results of 4th Mimicry Defense international elite competition.

Table 3
A summary of the results of the first phase of the penetration testing.

Attack phase	Attack description		IP-IP	CUTV MIN-MIN	SATCOM MIN-MIN
1. reconnaissance	Target		Host discoverable	Unable to find any hosts	Unable to find any hosts
	Ping scanning		Can be detected	Host undetectable	Host undetectable
	Operating system identification		The system fingerprint can be obtained	Host nonviable	Host nonviable
	Port scanning		Port services can be probed	Host nonviable	Host nonviable
2. Injected	Trojans implants	TCP Trojan	Connectable	Failed to make connection	Failed to make connection
		UDP Trojan	Connectable	Failed to make connection	Failed to make connection
		ICMP Trojan	Connectable	Failed to make connection	Failed to make connection
	Webshell		Connectable	Failed to make connection	Failed to make connection
3. Actions	ARP attack	Information sniffer	Target data sniffed	Failed to make connection	Failed to make connection
		Broken network attack	Target network outage	Failed to make connection	Failed to make connection

Table 4
Transaction fees for commissioners, verification, and ICV scoring contracts.

Transaction Type	Subtype	Gas Used	Gas Limit	Gas Price (Gwei)	Gas Price (ETH)	Gas Price ($)
Commissioners Contract	Sign Block	445,720	750,000	42.44	0.0189	0.0378
	Verify Claims	2,385,040	3,000,000	38.66	0.0923	0.1845
Verification Contract	Submit Claims	314,960	500,000	21.42	0.00675	0.0135
	Generate Proofs	22,050	50,000	10	0.00022	0.00044
	Verify Proofs	689,450	1,000,000	36.82	0.0254	0.0508
ICV Scoring	Honest ICVs	50,000	100,000	20	0.001	0.002
	Malicious ICVs	2,000,000	3,000,000	45.22	0.0904	0.1809
	Revoke ICV	500,925	750,000	29.37	0.0147	0.0294

and to enhance their scalability and mobility. MIN's management system is based on a consortium blockchain with voting consensus, which enables efficient management and support of the network by multiple nodes with high throughput.

As of this writing, MIN has undergone two phases of security testing. In the first phase, a rigorous penetration test was conducted in order to identify and patch potential security vulnerabilities in the network. In the second phase, MIN participated in international security contests to have its security features evaluated by security experts.

In the first phase of MIN's security testing, the process followed the standard Penetration Testing Execution Standard (PTES) and focused on information collection and analysis and penetration attacks. Information was collected and analyzed using various tools, including host discovery, ping scans, port scans, and operating system identifications. The penetration attacks stage included a variety of tests, including Trojan implantation, rebound shell attacks, Address Resolution Protocol (ARP) spoofing attacks, and man-in-the-middle attacks. The analysis involved scanning, sniffing, vulnerability identification, and man-in-the-middle attacks against the MIN network using tools such as Appscan, Wireshark, MSF, Nessus, and Arpspoof. Also, the team implanted and activated remote communication Trojan horses in order to test the isolation effect of the MIN network against traditional IP attack modes.

The MIN has been tested in two different scenarios. It was installed as the protocol for the office intranet system (CUTV) in order to meet the needs of daily office work. Not only did this implementation facilitate normal communication between hosts, but it also greatly improved network security by preventing most TCP/IP-based attacks and infiltration. Second, MIN was used for the integration of satellite communication (SATCOM). As part of the testing process, relevant satellite communication units were involved, which was a critical component of security in this scenario.

The results of the experiments are presented in Table 3. In Table 3, it is evident that MIN network has better security than the traditional network, and it can effectively protect against TCP/IP-based attacks and prevent Intranet attacks from spreading. An identity identification network has been implemented in BV-ICVs to prevent spoofing and tampering attacks. Every ICV is assigned a unique identity identifier based on the real identity information recorded in the consortium blockchain. ICVs must register with their real identity information and sign each packet with their private key in order to gain access to the network. In this way, any problematic behavior or resource usage can be traced accurately back to the ICV, allowing effective anti-traceability management and control. This also helps prevent impersonation and collusion attacks. At the application layer, the Multi-Identifier System (MIS) is used to verify the identity of each user, similar to how DNS is used in IP networks. By doing so, only authorized users are able to access the system and perform relevant operations based upon their permissions and privileges. MIS rejects any unauthorized access and record it in the tamper-proof ledger. By providing authentication and tracking capabilities for users, the MIS makes it difficult for unauthorized users to impersonate others or perform unauthorized actions.

In the second phase, the MIN network was provided to participants in the 4th Mimicry Defense International Elite Challenge for security attack testing[3]. Among the 3018 teams that entered the preliminary round of the competition, 48 international teams, including elite teams, universities, and teams from well-known enterprises, were selected to participate in the summit. Over 100 malicious files were uploaded to the server after 72 h of continuous attack penetration testing on the MIN system. At the end of the competition, no player was able to successfully breach the MIN network's security measures, as shown in Fig. 15.

7.5. Transaction fees analysis

The Table 4 below shows the transaction fees for different types of transactions in the commissioners, verification, and ICV scoring contracts. Gas is the unit used to measure the computational effort required to execute transactions on the Ethereum network. The gas used, the gas limit, and the gas price for each transaction are listed, along with the corresponding gas price in Gwei, ETH, and USD for each transaction, as currently 1 GWEI = 0.000000000112 ETH.

A commissioner's contract consists of two main types of transactions: signing blocks and verifying claims. "Verify Claims" requires the highest gas usage and gas price in comparison to the other transactions in this contract, suggesting that it is a complex and resource-intensive transaction. The reason for this is that claims submitted by the verifiers must undergo a complicated verification process. Multiple steps are involved in this verification process, such as verifying the authenticity of the claim, checking the validity of the proofs submitted, and ensuring that the claim meets all the requirements specified in the smart contract. This leads to high gas consumption, which incentivizes commissioners to vote for transactions by charging a high transaction fee.

The verification contract includes three subtypes of transactions, namely submitting claims, generating proofs, and verifying proofs. It is apparent that the "Verify Proofs" transaction requires the most gas usage and gas cost compared to the other transactions in this contract with a cost of 36.84 Gwei. The generate proofs operation, on the other hand, consumes less gas as it involves the generation of a cryptographic proof for each claim. Generating a proof involves performing some basic computations using cryptographic algorithms, which do not require a large amount of gas. Further, since the proof is generated offline and does not involve updating the blockchain, there is no need to perform expensive storage operations, which helps to reduce gas consumption.

ICV scoring consists of three subtypes of transactions, namely scoring honest ICVs, scoring malicious ICVs, and revoking ICVs. Based on its gas usage and gas price, the "Malicious ICVs" transaction is the most complex and resource-intensive transaction in this

[3] The 4th Mimicry Defense International Elite Challenge for security attack testinghttps://english.jschina.com.cn/23261/202111/t20211110_7306513.shtml and http://cessc.pmlabs.com.cn/disijie.html.

contract. Identifying and penalizing malicious ICVs requires a complex set of computations and storage operations. Typically, the process involves searching for relevant ICV records, checking them against the rules, and deducting points or rewards from the offending ICV's account. These computations and storage operations require significant computational resources, which translates into higher gas costs. Due to the critical nature of this operation, the higher gas limit may reflect the requirement for increased reliability and security.

Overall, these transaction fees are relatively expensive, with some fees exceeding $0.10 USD per transaction. Transactions are complicated due to the so-called 'contract invocation', which triggers the execution of one contract by the other and the need to incentivize miners to execute transactions.

8. Conclusion

To conclude, the proposed FL system based on blockchain, ZKPs, and LDP offers a practical solution that simultaneously achieves verifiability, privacy, and efficient communication while remaining practical and scalable. BV-ICVs allows the PoV consensus protocol to verify and vote for the integrity of trained local models, and it provides the ability for any party to verify the proof without having access to the prover's secret knowledge. However, BV-ICVs has limitations which points to future research opportunities. In order for the system to be fully decentralized, different decentralized consensus mechanisms can be explored, and more advanced privacy-preserving techniques, such as secure MPC or homomorphic encryption, may be required to safeguard the privacy of the participants and their data. Additionally, we are currently working on implementing our FL using different blockchain-based ZKPs such as bulletproof and STARKS to evaluate the most efficient method and hence improve our current system performance. Finally, while the proposed system is currently only applicable to Federated SVM, it can be adapted to other classes of ML protocols but with some effort. Overall, the proposed system provides a promising avenue for future research in FL and privacy-preserving techniques.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

, ZK-Rollups. URL: https://ethereum.org/en/developers/docs/scaling/zk-rollups/.

Alex, G., 2020. World's first practical hardware for zero-knowledge proofs acceleration. URL: https://blog.matter-labs.io/worlds-first-practical-hardware-for-zero-knowledge-proofs-acceleration-72bf974f8d6e.

An, C., Wu, C., 2020. Traffic big data assisted V2X communications toward smart transportation. Wirel. Networks 26, 1601–1610. https://doi.org/10.1007/s11276-019-02181-6.

Bachute, M.R., Subhedar, J.M., 2021. Autonomous driving architectures: Insights of machine learning and deep learning algorithms. Mach. Learn. Appl. 6, 100164. https://doi.org/10.1016/j.mlwa.2021.100164.

Ben Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M., 2014. Zerocash: Decentralized anonymous payments from Bitcoin. In: IEEE Symp. Secur. Priv., IEEE, Berkeley, CA, USA. pp. 459–474. https://doi.org/10.1109/SP.2014.36.

Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M., 2014. Succinct non-interactive zero knowledge for a von Neumann architecture. In: Proc. 23rd USENIX Conf. Secur. Symp., Usenix Association, San Diego, CA, pp. 781–796.

Bowe, S., Chiesa, A., Green, M., Miers, I., Tech, C., Mishra, P., Wu, H., 2019. ZEXE: enabling decentralized private computation. Technical Report. URL: https://eprint.iacr.org/2018/962.pdf.

H. Brendan, M., Eider Moore Daniel, R., Seth Hampson, B., Aguera, Y.A., 2017. Communication-efficient learning of deep networks from decentralized data. In: 20th Int. Conf. Artif. Intell. Stat., JMLR: W&CP, Fort Lauderdale, Florida, USA. pp. 1–10.

Bunz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G., 2018a. Bulletproofs: Short proofs for confidential transactions and more. In: Proc. - IEEE Symp. Secur. Priv., San Francisco, CA, USA, pp. 315–334. https://doi.org/10.1109/SP.2018.00020.

Bunz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G., 2018b. Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE Symp. Secur. Priv., IEEE. pp. 315–334. URL: https://ieeexplore.ieee.org/document/8418611/, https://doi.org/10.1109/SP.2018.00020.

Calvaresi, D., Calbimonte, J.P., Dubovitskaya, A., Mattioli, V., Piguet, J.G., Schumacher, M., 2019. The good, the bad, and the ethical implications of bridging blockchain and multi-agent systems. Information 10, 363. https://doi.org/10.3390/info10120363.

Campanelli, M., Fiore, D., Querol, A., 2019. LegoSNARK:: Modular design and composition of succinct zero-knowledge proofs. In: Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.. ACM, New York, NY, USA, pp. 2075–2092. https://doi.org/10.1145/3319535.3339820.

Chen, K., Xu, C., Liu, H., Wang, P., Chen, Z., 2022. Blockchain-based dangerous driving map data cognitive model in 5G–V2X for smart city security. Secur. Commun. Networks 2022, 1–10. https://doi.org/10.1155/2022/8922289.

Costan, V., Devadas, S., 2016. Intel SGX explained. Technical Report. URL: https://eprint.iacr.org/2016/086.pdf.

Damgård, I., Pastro, V., Smart, N., Zakarias, S., 2012. Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (Eds.), Adv. Cryptol. – CRYPTO 2012. Springer, Berlin Heidelberg, Berlin, Heidelberg, pp. 643–662. https://doi.org/10.1007/978-3-642-32009-5_38.

Daniel, B., Luís, B., Eran, T., 2019. ZKProof Community Reference. URL: https://docs.zkproof.org/pages/reference/reference.pdf.

Díaz, C., Seys, S., Claessens, J., Preneel, B., 2003. Towards measuring anonymity. In: Priv. Enhancing Technol. Springer, Berlin Heidelberg, pp. 54–68. https://doi.org/10.1007/3-540-36467-6_5.

Dwork, C., Roth, A., 2013. The Algorithmic foundations of differential privacy. Found. Trends Theor. Comput. Sci. 9, 211–407. https://doi.org/10.1561/0400000042.

Dwork, C., Smith, A., 2010. Differential privacy for statistics: What we Know and what we want to Learn. J. Priv. Confidentiality 1. https://doi.org/10.29012/jpc.v1i2.570.

Dwork, C., McSherry, F., Nissim, K., Smith, A., 2006. Calibrating noise to sensitivity in private data analysis. Theory Cryptogr., 265–284 https://doi.org/10.1002/jps.22559. arXiv:arXiv:1203.3453v5.

Eberhardt, J., Heiss, J., 2018. Off-chaining models and approaches to off-chain computations. In: Proc. 2nd Work. Scalable Resilient Infrastructures Distrib. Ledgers. ACM, New York, NY, USA, pp. 7–12. https://doi.org/10.1145/3284764.3284766.

Eli, B.S., Iddo, B., Yinon, H., Michael, R., 2019. Scalable, transparent, and post-quantum secure computational integrity. URL: https://eprint.iacr.org/2018/046.pdf.

Fang, C., Guo, Y., Ma, J., Xie, H., Wang, Y., 2022. A privacy-preserving and verifiable federated learning method based on blockchain. Comput. Commun. 186, 1–11. https://doi.org/10.1016/j.comcom.2022.01.002.

Fu, A., Zhang, X., Xiong, N., Gao, Y., Wang, H., Zhang, J., 2020. VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT. IEEE Trans. Ind. Informatics 1–1. https://doi.org/10.1109/TII.2020.3036166.

Garrido, G.M., Near, J., Muhammad, A., He, W., Matzutt, R., Matthes, F., 2021. Do I get the privacy I need? benchmarking utility in differential privacy libraries. Technical Report. URL: https://arxiv.org/abs/2109.10789, https://doi.org/10.48550/arXiv.2109.10789.

Goldreich, O., 2008. Foundations of cryptography: Volume 1, basic tools. 1st ed., Cambridge University Press.

Groth, J., 2016. On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J. (Eds.), Adv. Cryptol. – EUROCRYPT 2016. Springer, Berlin, Heidelberg, Berlin, Heidelberg, pp. 305–326. https://doi.org/10.1007/978-3-662-49896-5_11.

Guo, X., Liu, Z., Li, J., Gao, J., Hou, B., Dong, C., Baker, T., 2021. VeriFL: Communication-efficient and fast verifiable aggregation for federated learning. IEEE Trans. Inf. Forensics Secur. 16, 1736–1751. https://doi.org/10.1109/TIFS.2020.3043139.

Heiss, J., Eberhardt, J., Tai, S., 2019. From oracles to trustworthy data on-chaining systems. In: IEEE Int. Conf. Blockchain. IEEE, Atlanta, GA, USA, pp. 496–503. https://doi.org/10.1109/Blockchain.2019.00075.

Kang, J., Yu, R., Huang, X., Wu, M., Maharjan, S., Xie, S., Zhang, Y., 2019. Blockchain for secure and efficient data sharing in vehicular edge computing and networks. IEEE Internet Things J. 6, 4660–4670. https://doi.org/10.1109/JIOT.2018.2875542.

Li, H., Yang, X., 2021. Co-governed sovereignty network. Springer Singapore, Singapore. https://doi.org/10.1007/978-981-16-2670-8.

Li, H., Wu, J., Yang, X., Wang, H., Lan, J., Xu, K., Tan, H., Wei, J., Liang, W., Zhu, F., Lu, Y., Mow, W.H., Wai-Ho, Y., Zheng, Z., Yi, P., Ji, X., Liu, Q., Li, W., Tian, K., Zhu, J., Song, J., Liu, Y., Ma, J., Hu, J., Huang, J., Wei, G., Qi, J., Huang, T., Xing, K., 2020. MIN: Co-Governing multi-identifier network architecture and its prototype on operator's network. IEEE Access 8, 36569–36581. https://doi.org/10.1109/ACCESS.2020.2974327.

Li, K., Li, H., Wang, H., An, H., Lu, P., Yi, P., Zhu, F., 2020. PoV: An efficient voting-based consensus algorithm for consortium blockchains. Front. Blockchain 3. https://doi.org/10.3389/fbloc.2020.00011.

Li, T., Sahu, A.K., Talwalkar, A., Smith, V., 2020. Federated learning: Challenges, methods, and future directions. IEEE Signal Process. Mag. 37, 50–60. https://doi.org/10.1109/MSP.2020.2975749.

Li, Y., Chen, C., Liu, N., Huang, H., Zheng, Z., Yan, Q., 2021. A blockchain-based decentralized federated learning framework with committee consensus. IEEE Netw. 35, 234–241. https://doi.org/10.1109/MNET.011.2000263.

Matt, L., Corbin, P., 2017. The Keep network: A privacy layer for public blockchains. URL: https://coinpare.io/whitepaper/keep-network.pdf.

Nguyên, T.T., Xiao, X., Yang, Y., Hui, S.C., Shin, H., Shin, J., 2016. Collecting and analyzing data from smart device users with local differential privacy. CoRR abs/1606.05053. URL: http://dblp.uni-trier.de/db/journals/corr/corr1606.html#NguyenXYHSS16.

Noyes, C., 2016. Blockchain Multiparty Computation Markets at Scale. URL: https://www.overleaf.com/articles/blockchain-multiparty-computation-markets-at-scale/mwjgmsyybxvw.

Özdemir, V., Hekim, N., 2018. Birth of industry 5.0: Making sense of big data with artificial intelligence, the Internet of Things and next-generation technology policy. Omi. A J. Integr. Biol. 22, 65–76. https://doi.org/10.1089/omi.2017.0194. URL: http://www.liebertpub.com/doi/10.1089/omi.2017.0194.

Parno, B., Howell, J., Gentry, C., Raykova, M., 2013. Pinocchio: Nearly practical verifiable computation. In: IEEE Symp. Secur. Priv., IEEE, Berkeley, CA, USA. pp. 238–252. https://doi.org/10.1109/SP.2013.47.

Pokhrel, S.R., Choi, J., 2020. Federated learning with blockchain for autonomous vehicles: analysis and design challenges. IEEE Trans. Commun. 68, 4734–4746. https://doi.org/10.1109/TCOMM.2020.2990686.

Qi, Y., Hossain, M.S., Nie, J., Li, X., 2021. Privacy-preserving blockchain-based federated learning for traffic flow prediction. Futur. Gener. Comput. Syst. 117, 328–337. https://doi.org/10.1016/j.future.2020.12.003.

Qu, Y., Uddin, M.P., Gan, C., Xiang, Y., Gao, L., Yearwood, J., 2023. Blockchain-enabled federated learning: A Survey. ACM Comput. Surv. 55, 1–35. https://doi.org/10.1145/3524104.

Shrestha, R., Nam, S.Y., Bajracharya, R., Kim, S., 2020. Evolution of V2X communication and integration of blockchain for security enhancements. Electronics 9, 1338. https://doi.org/10.3390/electronics9091338.

Simunic, S., Bernaca, D., Lenac, K., 2021. Verifiable computing applications in blockchain. IEEE Access 9, 156729–156745. https://doi.org/10.1109/ACCESS.2021.3129314.

Taheri, R., Javidan, R., Shojafar, M., Pooranian, Z., Miri, A., Conti, M., 2020. On defending against label flipping attacks on malware detection systems. Neural Comput. Appl. 32, 14781–14800. https://doi.org/10.1007/s00521-020-04831-9.

Tian, D., Gong, W., Liu, W., Duan, X., Zhu, Y., Liu, C., Li, X., 2018. Applications of intelligent computing in vehicular networks. J. Intell. Connect. Veh. 1, 66–76. https://doi.org/10.1108/JICV-01-2018-0001.

Tolpegin, V., Truex, S., Gursoy, M.E., Liu, L., 2020. Data Poisoning Attacks Against Federated Learning Systems. In: Liqun, C., Ninghui, L., Kaitai, L., Steve, S. (Eds.), Eur. Symp. Res. Comput. Secur.. Springer, Guildford, UK, pp. 480–501. https://doi.org/10.1007/978-3-030-58951-6_24.

Toyoda, K., Zhao, J., Zhang, A.N.S., Mathiopoulos, P.T., 2020. Blockchain-enabled federated learning with mechanism design. IEEE Access 8, 219744–219756. https://doi.org/10.1109/ACCESS.2020.3043037.

Wang, Q., Guo, Y., Wang, X., Ji, T., Yu, L., Li, P., 2020. AI at the edge: Blockchain-empowered secure multiparty learning with heterogeneous models. IEEE Internet Things J. 7, 9600–9610. https://doi.org/10.1109/JIOT.2020.2987843.

Wang, B., Han, Y., Wang, S., Tian, D., Cai, M., Liu, M., Wang, L., 2022. A Review of intelligent connected vehicle cooperative driving development. Mathematics 10, 3635. https://doi.org/10.3390/math10193635.

Wenxiu, D., Wei, S., Zheng, Y., Robert, H.D., 2021. An efficient and secure scheme of verifiable computation for Intel SGX. Technical Report. URL: https://arxiv.org/pdf/2106.14253.pdf.

Xu, G., Li, H., Liu, S., Yang, K., Lin, X., 2020. VerifyNet: Secure and verifiable federated learning. IEEE Trans. Inf. Forensics Secur. 15, 911–926. https://doi.org/10.1109/TIFS.2019.2929409.

Zhu, S., Li, R., Cai, Z., Kim, D., Seo, D., Li, W., 2022. Secure verifiable aggregation for blockchain-based federated averaging. High-Confidence Comput. 2, 100046. https://doi.org/10.1016/j.hcc.2021.100046.

Zyskind, G., 2018. Chapter 15 Enigma: Decentralized computation platform with guaranteed privacy. In: New Solut. Cybersecurity. The MIT Press, pp. 425–454. https://doi.org/10.7551/mitpress/11636.003.0018. URL: https://direct.mit.edu/books/book/3582/chapter/120163/enigma-decentralized-computation-platform-with.